

基于混合效应模型的 NIPT 时点优化与胎儿异常判定

摘要

无创产前检测 (NIPT) 结果的准确性高度依赖于胎儿游离 DNA 浓度, 该浓度受孕周 (GA)、身体质量指数 (BMI) 等因素显著影响。本文基于临床数据, 构建了混合效应模型与孕妇潜在风险最小的优化模型相结合的优化框架, 系统研究 NIPT 检测时点优化与胎儿染色体异常判定问题。

针对问题一, 首先对胎儿 Y 染色体浓度 (FF) 与孕周 (GA)、身体质量指数 (BMI) 的相关性进行研究。通过对 FF 进行 logit 变换, 建立包含二次项的线性混合效应模型 (LMM), 并引入随机截距与斜率捕捉个体差异。结果表明: FF 随 GA 单调上升, BMI 呈倒 U 型关系。经过似然比检验, GA 一次项与二次项均显著, 而 BMI 仅二次项显著, 模型拟合优度达 $R^2 = 0.8962$, 显著优于传统广义线性模型。

针对问题二, 本研究以男胎 Y 染色体浓度最早达标时间 ($FF \geq 4\%$) 为核心, 构建孕妇潜在风险最小化的优化模型, 充分考虑孕妇检测失败风险、时间风险, 利用动态规划算法对 BMI 进行 5 段分组并确定最优检测时点。分组结果为: $[20.7, 26.8] \rightarrow 11.9$ 周、 $[26.9, 39.9] \rightarrow 12.8$ 周、 $[40.0, 42.5] \rightarrow 17.5$ 周、 $[42.8, 46.8] \rightarrow 20.4$ 周、 $[44.7, 46.8] \rightarrow 22.8$ 周。检测误差分析表明, 高 BMI 群体对误差更敏感, 推荐时点随误差增大而推迟, 但分组结构保持稳定。

针对问题三, 本研究引入年龄、怀孕次数、生产次数等多因素, 建立扩展孕妇潜在风险最小化的优化模型。通过动态规划分段和单调性控制, 得到 5 组年龄层, 各 5 组 BMI 分区及对应时点。多因素协同效应显著, 年龄与怀孕次数增加使推荐时点提前, 生产次数增加则推迟检测, 经交叉验证与灵敏度回验验证稳健。

针对问题四, 本研究针对女胎染色体异常判定, 提出“男胎校准 + 协变量校正 + 经验零分布 + 多重检验校正”的综合判定模型。通过孕周分层、Z 值标准化与 FDR 控制, 输出三级判定结果 (阴性/可疑/阳性), 并叠加 ChrX 质控门限与一致性聚合, 降低批次效应。在 1% 假阳性率约束下, 女胎检出 3 例阳性、26 例可疑, 验证了模型具有良好的稳健性与临床适用性。

本文创新性在于: 一是建立了融合混合效应与动态规划的 NIPT 时点优化框架; 二是提出了多因素协同下的分组策略与误差传播分析方法; 三是制作了交互网页, 为孕妇提供最优 NIPT 检测时间, 切实推广模型应用。

关键字: 无创产前检测 混合效应模型 优化模型 Z 值判定 染色体异常筛查

一、问题重述

1.1 问题背景

近年来我国的无创产前检测技术已实现广泛应用与持续迭代,无创产前检测 (NIPT) 是一项基于孕妇外周血中胎儿游离 DNA 的高通量测序技术,用于评估胎儿常见染色体非整倍体异常^[1,2]。该技术主要针对唐氏综合征 (21 三体)、爱德华氏综合征 (18 三体) 及帕陶氏综合征 (13 三体) 等染色体疾病进行筛查^[2-4]。NIPT 结果的准确性在很大程度上依赖于胎儿分数/性染色体相关的浓度指标: 对于男胎, 通常认为当胎儿游离 DNA 所致的 Y 染色体信号达到一定比例 (临床上常以 4% 作为经验阈值) 时, 检测结果更为可靠; 对于女胎, 则需确保 X 染色体相关指标处于正常范围^[5-7]。此外, 孕妇的孕周 (GA) 及身体质量指数 (BMI) 等因素对检测性能与最佳检测时机具有显著影响——胎儿分数随孕周增加而上升, 而高 BMI 与低胎儿分数相关^[5,6,8]。临床实践中, 过早检测可能导致 “信息不足/不报告 (no-call)”, 而过晚检测则与母婴随访与干预时机相关风险上升相关^[3,4,9]。因此, 基于生物学规律与统计证据, 科学确定检测时点, 对于在保证检出能力的同时降低临床风险具有重要意义^[7,10]。

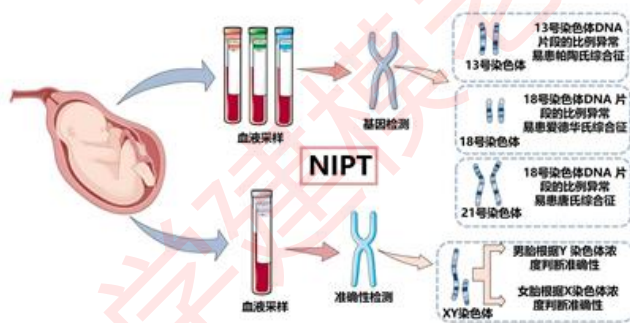


图 1 NIPT 示意图

1.2 问题要求

本研究需结合所提供的临床数据, 建立数学模型以系统研究以下四个问题:

问题 1 分析胎儿 Y 染色体浓度与孕妇孕周数、BMI 等指标之间的相关性, 建立关系模型并检验其显著性。

问题 2 以男胎孕妇的 BMI 为主要影响因素, 对其进行合理分组, 确定每组的最佳 NIPT 检测时点, 以最小化孕妇的潜在风险, 并分析检测误差对结果的影响。

问题3 综合考虑体重、年龄、检测误差及 Y 染色体浓度达标比例等因素，对男胎孕妇进行 BMI 分组，确定每组的最佳 NIPT 时点，以最小化潜在风险，并分析误差影响。

问题4 针对女胎孕妇，综合考虑 X 染色体及其他染色体（21、18、13 号）的 Z 值、GC 含量、读段数、比例及 BMI 等因素，建立女胎染色体异常的判定方法。

1.3 我们的工作

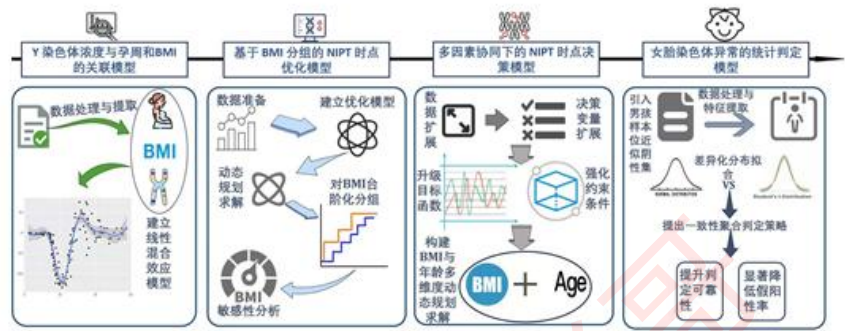


图 2 总体流程图

二、模型假设

为简化问题，本文做出以下假设：

- **假设 1：**假设附件提供的 NIPT 临床数据真实准确。
- **假设 2：**假设在相近的 BMI 值区间内，孕妇的生理特性及胎儿 Y 染色体浓度的变化规律是连续且平滑的。
- **假设 3：**假设模型中的测量误差服从均值为零的正态分布。

三、符号说明

符号	说明	单位
FF	胎儿 Y 染色体浓度	—
GA	孕周	周
BMI	身体质量指数	kg/m^2
y	$\text{logit}(FF)$	—
$\beta_0, \beta_1, \dots$	固定效应系数	—
u_i, b_i	随机效应	—
σ^2	方差	—
$p(t, b)$	达标概率	—
θ	浓度阈值 (0.04)	—
$T_*(b)$	最早达标时间	周
κ	误差放缩因子	—
r_{fail}	检测失败风险	—
$\phi(t)$	临床时间风险	—
Z	Z 值	—
q	FDR 校正后的 q 值	—

四、问题一：Y 染色体浓度与孕周、BMI 的关联模型

4.1 问题分析

问题一的核心目标是建立胎儿 Y 染色体浓度 (FF) 与孕周 (GA) 及身体质量指数 (BMI) 之间的定量关系模型。FF 作为 (0,1) 区间的比例变量，其分析包含两个要点：一是需要处理同一孕妇多次测量导致的组内相关性；二是其次，数据具有典型的纵向结构，同一孕妇的多次观测在生理上必然相关。本研究对因变量 FF 进行 logit 变换以消除比例数据的边界约束；对孕周和 BMI 引入二次项以捕捉非线性关系；采用线性混合效应模型框架处理纵向数据的组内相关性，通过随机截距和随机斜率刻画个体异质性，从而构建出数学上严谨且生物学上合理的定量模型。

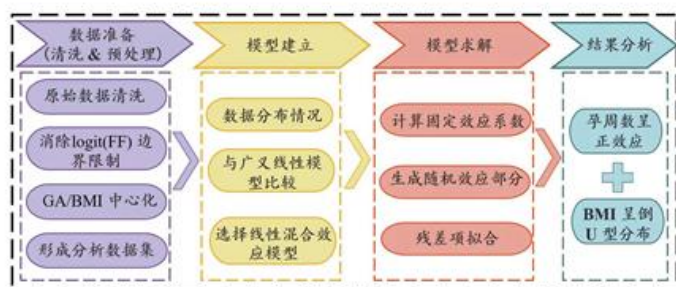


图3 问题一流程图

4.2 数据预处理

本研究使用来自真实临床检测记录的 NIPT 数据，有效观测量 $N = 1082$ ，涉及 267 名孕妇，人均测量次数约为 4.05。核心变量统计如下：

表1 核心变量的描述性统计结果

变量	均数 ($\bar{x}$)	标准差 (s)
FF	0.0772	0.0335
GA	16.85	4.08
BMI	32.29	2.97

为直观展示 FF 与 GA、BMI 之间的复杂关系模式，本节构建二维热力图进行可视化分析。将 GA 和 BMI 分别离散化为若干区间，计算每个 (GA, BMI) 组合下 FF 的均值，生成热力图矩阵。

热力图分析结果：

非单调性：FF 值在 GA-BMI 平面上呈现明显的非单调变化模式，排除了简单线性关系的可能性；

局部最优区域：热力图显示 FF 在特定的 (GA, BMI) 区域内达到相对高值，呈现“山峰”状分布特征

该可视化结果从数据层面证实了变量间关系的复杂性和非线性特征，为后续引入二次项和交互项提供了直观的经验证据，同时排除了使用简单线性模型的可行性。



其中：

(1) 固定效应部分：

$$\eta_{fix} = \beta_0 + \beta_1 GA_{c,ij} + \beta_2 GA_{c,ij}^2 + \beta_3 BMI_{c,ij} + \beta_4 BMI_{c,ij}^2 \quad (4)$$

该部分用于刻画群体层面 FF 随 GA 与 BMI 变化的平均趋势。引入二次项旨在捕捉变量间可能存在的非线性关系（如增速变化、倒 U 型关系等），从而更精确地揭示其内在相关性。

(2) 随机效应部分：

$$u_i \sim \mathcal{N}(0, \sigma_u^2), \quad b_i \sim \mathcal{N}(0, \sigma_b^2), \quad \text{Cov}(u_i, b_i) = \sigma_{ub} \quad (5)$$

该部分用于描述个体层面的随机变异。其中，随机截距 u_i 反映第 i 位孕妇 FF 的基线水平差异；随机斜率 b_i 反映其对孕周变化的响应速率差异。该结构有效解决了同一孕妇多次测量带来的组内相关问题。

(3) 残差项：

$$\varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2) \quad (6)$$

代表模型未能解释的随机测量误差。

该模型兼具固定效应与随机效应，既能够分析 GA 和 BMI 对 FF 的群体平均影响，又能合理处理纵向数据中的个体差异与重复测量相关性，从而构建出稳健且解释性强的统计关系模型。

4.3.3 模型的比较与选择

为确定最优模型形式，本节系统比较了线性混合效应模型和种广义线性模型。评价指标涵盖 AIC、对数似然值以及在 FF 原量纲上的拟合优度（ R^2 和 RMSE）。

表 2 线性混合效应模型与广义线性模型拟合优度对比

模型	AIC	对数似然	$R^2(\text{FF})$	RMSE(FF)	MASE
MixedLM: 随机截距 + 斜率 + 二次项	712.63	-347.32	0.8962	0.0108	0.322
GLM	749.92	-367.96	0.0433	0.0113	0.328

基于前述模型比较结果，最终确定了包含二次项和随机斜率的混合效应模型作为最优模型。该模型不仅具有最低的 AIC 值（712.63）和最优的对数似然值（-347.32），在 FF 原量纲上也表现出最佳的拟合优度（ $R^2 = 0.8962$, RMSE=0.0108）。与广义线性模型相比，线性混合效应模型带来了显著的改善（似然比检验 $LR=41.29$, $p \approx 1.1 \times 10^{-9}$ ），强有力地证明了捕捉非线性关系的必要性。

4.4 模型求解

本节采用极大似然估计方法对所定义的线性混合效应模型进行参数估计。具体算法步骤如下：

Step 1: 模型表达与似然函数构建

将模型写为矩阵形式：

$$\mathbf{y} = \mathbf{X}\beta + \mathbf{Z}\mathbf{u} + \varepsilon \quad (7)$$

其中， $\mathbf{y}$ 为所有观测的 logit(FF) 向量； $\mathbf{X}$ 为固定效应设计矩阵，包含 GA_c 、 GA_c^2 、 BMI_c 、 BMI_c^2 ； $\mathbf{Z}$ 为随机效应设计矩阵； $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{G})$ 为随机效应向量； $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ 为残差向量。总方差-协方差结构为：

$$\text{Var}(\mathbf{y}) = \mathbf{V} = \mathbf{Z}\mathbf{G}\mathbf{Z}^T + \mathbf{R} \quad (8)$$

对应的受限对数似然函数 (REML) 为：

$$\ell(\beta, \theta) = -\frac{1}{2} \log |\mathbf{V}| - \frac{1}{2} \log |\mathbf{X}^T \mathbf{V}^{-1} \mathbf{X}| - \frac{1}{2} (\mathbf{y} - \mathbf{X}\beta)^T \mathbf{V}^{-1} (\mathbf{y} - \mathbf{X}\beta) \quad (9)$$

其中 θ 为方差组分参数。

Step 2: 迭代优化求解

使用牛顿-拉弗森算法或期望最大化算法 (EM) 进行迭代优化，逐次更新固定效应系数 β 与方差组分 θ (包括 $\sigma_u^2, \sigma_b^2, \sigma_{ub}, \sigma^2$)，直至对数似然函数收敛。

Step 3: 结果提取与解释

迭代收敛后，提取各参数的估计值、标准误及假设检验结果 (见表 3)。为更直观解释模型结果，将 logit 尺度下的固定效应系数转换为 FF 原尺度下的平均边际效应 (Average Marginal Effect, AME)，具体通过数值差分或 Delta 方法实现：

$$\text{AME}_{GA} = \frac{1}{n} \sum_{i=1}^n \frac{\partial \text{FF}}{\partial GA} \approx \frac{1}{n} \sum_{i=1}^n \beta_1 \cdot \text{FF}_i \cdot (1 - \text{FF}_i) \quad (10)$$

Step 4: 随机效应估计

通过 BLUP (Best Linear Unbiased Prediction) 估计每位孕妇的随机效应 u_i 和 b_i ，并计算组内相关系数 ICC：

$$\text{ICC} = \frac{\sigma_u^2 + \sigma_b^2 \cdot \text{Var}(GA)}{\sigma_u^2 + \sigma_b^2 \cdot \text{Var}(GA) + \sigma^2} \quad (11)$$

4.6 结论

本研究通过建立线性混合效应模型，成功揭示了胎儿 Y 染色体浓度与孕周、BMI 之间的定量关系，主要结论如下：

1. **孕周效应**：FF 随孕周显著单调上升 ($\beta_{GA} = 0.042, p < 0.001$)，且存在加速效应 ($\beta_{GA^2} = 0.003, p < 0.001$)，孕周每增加 1 周，FF 平均上升 0.316 个百分点
2. **BMI 效应**：FF 与 BMI 呈倒 U 型关系 ($\beta_{BMI^2} = -0.003, p = 0.009$)，在 BMI ≈ 31 kg/m² 处达到峰值，之后随 BMI 增加而下降
3. **个体差异**：个体间变异显著 (ICC ≈ 0.81)，证实采用混合效应模型的必要性

五、问题二：基于 BMI 分组的 NIPT 时点优化模型

5.1 问题分析

NIPT 检测的准确性高度依赖于胎儿 Y 染色体浓度是否达到临床阈值 (4%)，而该浓度受孕妇 BMI 和孕周的共同影响。临床实践表明，BMI 较高的孕妇往往需要更长的等待时间才能使胎儿 Y 染色体浓度达标。因此，科学确定不同 BMI 群体对应的最佳检测时点，对最小化检测失败风险与临床时间风险具有重要意义。本文旨在建立一种基于 BMI 分组的 NIPT 时点优化模型，通过量化 BMI 与最早达标时间的关系，进行合理分组并在每组内推荐统一检测时间，从而在保障检测可靠性的前提下，尽可能降低因检测过早或过晚所带来的潜在风险。

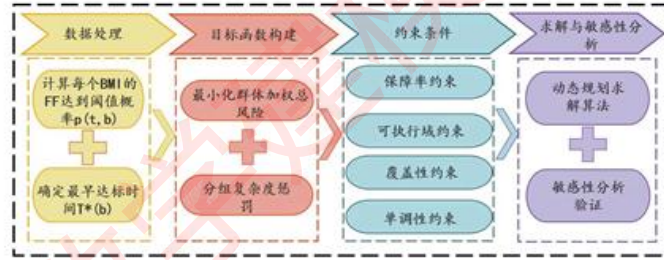


图 7 问题二流程图

5.2 数据处理

基于问题一的线性混合效应模型，在孕周区间 $[10, 28]$ 的离散网格上，对每个 BMI 水平逐点计算 FF 达到阈值 $\theta = 0.04$ 的概率 $p(t, b)$ ，并以首次使该概率不低于 $q = 0.95$ 的孕周作为最早达标时间 $T^*(b)$ ；随后将离散的 $\{(b, T^*(b))\}$ 输入带单调先验的贝叶斯单调

回归进行平滑, 约束曲线随 BMI 单调不降, 得到稳健的平滑函数 $T_{\text{Bayes}}(b)$ 及其 95% 置信带, 作为后续 BMI 分组与台阶化优化的输入。

5.3 模型建立

5.3.1 基于期望损失最小化的 BMI 分组与时点决策

为将连续的推荐孕周曲线转化为临床可执行的有限分组策略, 需同步优化分组边界和分组的统一推荐检测时点。本小节详细阐述该优化模型的目标函数与约束条件。

1. 决策变量

设将 BMI 定义域划分为 K 段(本研究取 $K = 5$), 定义分段断点集合 $\mathbf{c} = (c_0, c_1, \dots, c_K)$, 满足 $c_0 = b_{\min}, c_K = b_{\max}, c_{s-1} < c_s$ 。

定义每段的统一推荐检测孕周为 $\mathbf{t} = (t_1, t_2, \dots, t_K)$, 其中 $t_s \in [12, 28]$ 。

若某孕妇的 BMI 值 $b_i \in (c_{s-1}, c_s]$, 则其被归入第 s 组, 并采用统一的推荐检测时点 t_s 。

2. 风险度量优化目标需最小化总体风险, 该风险由两部分构成:

技术失败风险 $r_{\text{fail}}(t, b)$: 在推荐孕周 t 进行抽血检测时, 胎儿 Y 染色体浓度仍未达到阈值 $\theta = 4\%$ 的概率。该风险是孕周 t 和 BMI b 的函数:

$$r_{\text{fail}}(t, b) = 1 - p_{\kappa}(t, b) \quad (12)$$

其中 $p_{\kappa}(t, b)$ 由问题一的混合效应模型推得, 并引入了误差缩放因子 κ 以考量检测噪声。

时间风险 $\phi(t)$: 检测时机过晚所带来的临床代价。该函数被定义为分段线性函数, 以体现不同孕周阶段风险的差异:

$$\phi(t) = \begin{cases} 0, & t \leq 12 \\ 12\alpha(t - 12), & 12 < t < 28 \\ 2816\alpha + \beta(t - 28), & t \geq 28 \end{cases} \quad (13)$$

其中, α 和 β 为惩罚系数, 且通常 $\beta > \alpha$, 表示孕 28 周后延迟检测的临床风险代价更高。此函数迫使优化解不会为了无限提高达标概率而过度推迟检测。

5.3.2 目标函数

目标函数旨在最小化所有孕妇的群体加权总风险, 并附加对分组复杂度的正则化惩罚:

$$\min_{\mathbf{c}, \mathbf{t}} \mathcal{J}(\mathbf{c}, \mathbf{t}) = \sum_{s=1}^K \sum_{i \in I_s} w_i [r_{\text{fail}}(t_s, b_i) + \phi(t_s)] + \lambda K \quad (14)$$

其中:

I_s 表示被划分到第 s 组的个体索引集合。

w_i 代表第 i 个样本的权重 (可反映其在人群中的分布), 满足 $\sum_i w_i = 1$ 。

λK 是正则化项, 用于抑制分组数 K 过多导致的方案过于复杂、难以临床执行。当 K 固定时, 可取 $\lambda = 0$ 。

该目标函数体现了“安全-及时-可执行”三者的权衡: r_{fail} 追求检测的可靠性 (安全); $\phi(t)$ 约束检测的及时性; λK 保证策略的简洁性 (可执行)。

5.3.3 约束条件

优化问题需满足以下约束:

保障率约束 (安全性): 对于任意分组 s , 其组内在最不利情况下的达标概率仍需满足最低置信水平 q (基线 $q = 0.95$)。

$$\min_{i \in I_s} p_K(t_s, b_i) \geq q, \quad s = 1, \dots, K \quad (15)$$

该约束确保了分组的鲁棒性, 即只要孕妇属于该组, 即保证其在推荐时点检测的可靠性。

可执行域约束: 推荐的检测孕周必须在临床可行的时间窗口内。

$$t_s \in [12, 28], \quad s = 1, \dots, K \quad (16)$$

覆盖性与非重叠性约束: 分组应覆盖整个 BMI 定义域, 且各区间不重叠。

$$b_{\min} = c_0 < c_1 < \dots < c_K = b_{\max} \quad (17)$$

单调性约束 (临床合理性): 推荐孕周应随 BMI 增加而非递减。

$$t_1 \leq t_2 \leq \dots \leq t_K \quad (18)$$

该约束符合“高 BMI 倾向更晚检测”的临床经验, 并能减少分组边界处的策略抖动, 增强方案的一致性。

5.4 模型求解

本研究采用离散化结合动态规划 (DP) 的算法求解上述优化模型, 具体步骤如下:

Step1: 离散化处理 将 BMI 取值范围按步长 (如 0.1 kg/m^2) 离散化为有序网格点 $b_{(1)} \leq b_{(2)} \leq \dots \leq b_{(n)}$ 。将孕周搜索区间 $[10, 28]$ 按步长 (如 0.1 周) 离散化为候选检测时点集合 $\mathcal{T}$ 。

Step2: 预计算区间代价 对于任意 BMI 区间 $[i: j]$ 和候选时点 $t \in \mathcal{T}$, 若满足该区间内所有 BMI 点在时点 t 的最小达标概率约束 (即 $\min_{k \in [i: j]} p_K(t, b_{(k)}) \geq q$), 则计算将该区间归为一组并推荐在 t 周检测的代价:

$$C(i, j, t) = \sum_{k=i}^j w_{(k)} [r_{\text{fail}}(t, b_{(k)}) + \phi(t)] \quad (19)$$

对该区间，其最小代价为 $C(i, j) = \min_{t \in \mathcal{T}} C(i, j, t)$ ，并记录对应的最优时点 $t^*(i, j)$ 。若不满足约束，则置 $C(i, j) = +\infty$ 。

Step3: 动态规划分段定义 $DP[j, s]$ 为将前 j 个 BMI 点划分为 s 段的最小总风险。状态转移方程为：

$$DP[j, s] = \min_{0 \leq i < j} \{DP[i, s-1] + C(i+1, j)\} \quad (20)$$

同时记录状态转移路径。算法复杂度为 $O(n^2)$ 。

Step4: 回溯与后处理 从 $DP[n, K]$ 开始回溯，得到最优的分组断点 $c_0, c_1, \dots, c_K$ 以及每段对应的推荐检测时点 $t_s = t^*(c_{s-1} + 1, c_s)$ 。为确保分组的单调性和保障率约束，对得到的时点序列进行投影和微调（如整体平移）。

Step5: 结果输出 最终输出 K 个 BMI 区间及其对应的推荐检测孕周、段内平均保障率、平均风险等指标。

该算法高效地解决了分组和时点指派联合优化问题，确保了在满足临床约束下的全局风险最小化。

5.5 求解结果

为便于理解，将连续建议曲线与台阶化策略进行对照展示，并给出各分段的统计汇总。

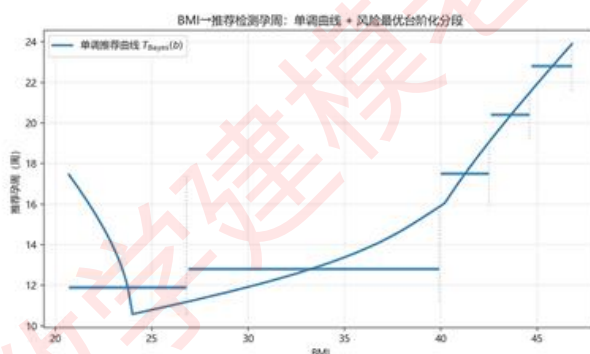


图8 连续建议 $T_{\text{Bayes}}(b)$ 与5段风险最优台阶化的叠加。

表 4 不同 BMI 分组的推荐孕周

组别	BMI 区间	推荐孕周 (周)	保障率均值	风险均值
1	[20.7, 26.8]	11.9	0.9103	0.0897
2	[26.9, 39.9]	12.8	0.9693	0.0547
3	[40.0, 42.5]	17.5	0.9244	0.2406
4	[42.8, 46.8]	20.4	0.9264	0.3256
5	[44.7, 46.8]	22.8	0.9243	0.3997

5.6 敏感性分析

为评估推荐策略对检测噪声的稳健性, 针对“误差缩放因子 κ ”开展灵敏度分析, 并在两种保障水平下统计加权平均推荐孕周及其相对位移。表6给出了真实测算结果, 噪声越大、保障越高, 平均推荐孕周越晚。

表 5 不同 κ 下的平均推荐孕周与相对位移

情景	κ	段数	平均推荐孕周 (周)	相对位移 Δt (周)
$q = 0.95$	0.75	5	18.44	+1.36
$q = 0.95$	1.00	5	17.08	+0.00
$q = 0.95$	1.25	5	17.73	+0.65
$q = 0.95$	1.50	5	19.67	+2.59
$q = 0.98$	0.75	5	17.30	-1.32
$q = 0.98$	1.00	5	18.62	+0.00
$q = 0.98$	1.25	5	21.38	+2.76
$q = 0.98$	1.50	5	23.10	+4.48

表 6 不同 κ 下的平均推荐孕周与相对位移

情景	κ	段数	平均推荐孕周 (周)	相对位移 Δt (周)
$q = 0.95$	0.75	5	18.44	+1.36
$q = 0.95$	1.00	5	17.08	+0.00
$q = 0.95$	1.25	5	17.73	+0.65
$q = 0.95$	1.50	5	19.67	+2.59
$q = 0.98$	0.75	5	17.30	-1.32
$q = 0.98$	1.00	5	18.62	+0.00
$q = 0.98$	1.25	5	21.38	+2.76
$q = 0.98$	1.50	5	23.10	+4.48

六、问题三：多因素协同下的 NIPT 时点决策模型

6.1 问题分析

在问题二仅考虑 BMI 单一因素的基础上，问题三进一步引入孕妇年龄 (AGE)、怀孕次数 (G) 与生产次数 (P) 三项临床协变量，旨在更全面地评估多因素对胎儿 Y 染色体浓度 (FF) 最早达标时间的影响。需要解决的核心问题包括：如何筛选具有统计显著性的协变量；如何建立多因素混合效应模型；如何在兼顾达标比例与检测误差的前提下，给出基于 BMI 分组的可执行推荐策略；以及如何评估模型在不同误差口径下的稳健性。

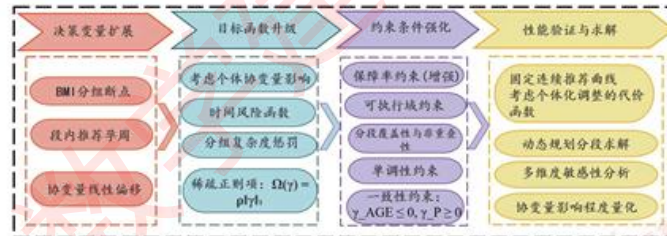


图 9 问题三流程图

6.2 模型建立

6.2.1 扩展的混合效应模型

首先在问题一模型基础上引入多因素，建立扩展的混合效应模型。对 FF 进行 logit 变换后，模型形式为：

$$\begin{aligned} y_{ij} = & \beta_0 + \beta_1 GA_{c,ij} + \beta_2 GA_{c,ij}^2 + \beta_3 BMI_{c,ij} \\ & + \beta_4 BMI_{c,ij}^2 + \beta_{AGE} AGE_{c,ij} + \beta_G G_{c,ij} \\ & + \beta_P P_{c,ij} + u_{0i} + u_{1i} GA_{c,ij} + \varepsilon_{ij} \end{aligned} \quad (21)$$

其中新引入的固定效应项：(1) β_{AGE} : 年龄的偏效应 (2) β_G : 怀孕次数的偏效应 (3) β_P : 生产次数的偏效应

通过 Elastic-Net 变量筛选与显著性检验，确保上述纳入模型的协变量具有统计学意义。

6.2.2 多因素优化模型框架

在问题二优化模型基础上，引入协变量向量：

$$z = (AGE_c, G_c, P_c)^T$$

其中各变量均已中心化处理。目标是在给定概率模型的前提下，构造可执行的分段推荐策略，并最小化总体风险。

决策变量 (1) 分段断点: $c = (c_0, c_1, \dots, c_K)$, 满足 $c_0 = b_{\min}, c_K = b_{\max}$ (2) 段内推荐孕周: $t = (t_1, \dots, t_K)$, $t_s \in [12, 28]$ (3) **新增**: 协变量线性偏移: $\gamma \in \mathbb{R}^3$, 用于对个体推荐时点进行个性化微调

目标函数

$$\min_{c, t, \gamma} J(c, t, \gamma) = \sum_{s=1}^K \sum_{i \in \mathcal{I}_s} w_i [r_{\text{fail}}(t_s + \gamma^T z_i, b_i; z_i) + \phi(t_s + \gamma^T z_i)] + \lambda K + \Omega(\gamma) \quad (22)$$

其中: r_{fail} 为技术失败风险, 考虑了个体协变量影响 - ϕ 为时间风险函数 - **新增**: $\Omega(\gamma) = \rho \|\gamma\|_1$ 为稀疏正则项, 防止对协变量的过度依赖

约束条件 1. 保障率约束:

$$\min_{i \in \mathcal{I}_s} \min_{\kappa \in \mathcal{K}} p_{\kappa}(t_s + \gamma^T z_i, b_i; z_i) \geq q, \quad s = 1, \dots, K$$

同时考虑段内最弱个体与最差检测条件, 安全性更强

2. 可执行域约束:

$$t_s \in [12, 28], \quad \|\gamma\|_\infty \leq \Gamma$$

相较问题二新增约束: 对协变量偏移量施加界约束, 保证调整幅度在临床可接受范围内

3. 分段覆盖性与非重叠性

4. 单调性约束: $t_1 \leq t_2 \leq \dots \leq t_K$

5. 新增——一致性约束: 同一段内个体推荐时点的调整方向需符合临床先验: (1) 年龄效应: $\gamma_{AGE} \leq 0$ (年龄越大, 推荐时点越早) (2) 生产次数效应: $\gamma_P \geq 0$ (生产次数越多, 推荐时点越晚)

6.3 模型求解

采用“两阶段 动态规划”策略:

1. 偏移量预估计: 固定一条连续推荐曲线, 求解:

$$\hat{\gamma} = \arg \min_{\|\gamma\|_\infty \leq \Gamma} \sum_i w_i [r_{\text{fail}}(T_{\text{mono}}(b_i) + \gamma^T z_i, b_i; z_i)] + \Omega(\gamma)$$

2. 区间代价预计算: 对每个 BMI 区间和候选时点, 计算带有个体化调整的代价函数

3. 动态规划分段: 采用与问题二相同的 DP 框架, 但代价函数考虑了个体协变量调整

4. 后处理与稳健性回验: 对多种误差场景和保障水平进行回验, 必要时进行整体平移

算法复杂度保持在 $O(n^2)$ 量级, 与问题二相同, 具备工程可行性。

6.4 求解结果

本节对样本按年龄分层并在各层内进行 BMI 分段台阶化。表 7 显示出清晰的一致性规律:

- 同龄层内随 BMI 单调后移。各年龄层的推荐孕周均随 BMI 增加而后移。其中在 BMI ≈ 40 左右常出现阶跃式上移, 提示高 BMI 区间对时点安排更敏感。
- 同一 BMI 水平下随年龄整体验证“更晚”。在中高 BMI 段, 年龄越大推荐孕周越晚。
- 意义。低 BMI 且年轻 (如 <25 岁、BMI <32) 的受检者可在早孕期 (10–13 周) 安排检测; BMI 升高与/或年龄升高时, 需后移至中晚孕期 (例如 BMI ≈ 40 时普遍需要 18–22 周, ≥ 40 岁且 BMI > 34 时建议 25–26 周附近)。

表 7 按年龄分层的 BMI 分段推荐孕周

年龄层	BMI 区间	推荐孕周 (周)	网格数	年龄层	BMI 区间	推荐孕周 (周)	网格数
<25 岁	[28.98, 31.98]	10.6	31	30-34 岁	[35.50, 41.50]	15.0	61
<25 岁	[32.08, 34.48]	12.8	25	30-34 岁	[41.60, 44.10]	18.6	26
<25 岁	[34.58, 36.08]	15.2	16	30-34 岁	[44.20, 46.80]	22.1	27
<25 岁	[36.18, 37.58]	18.5	15	35-39 岁	[27.92, 28.92]	17.9	11
<25 岁	[37.68, 39.28]	21.5	17	35-39 岁	[29.02, 32.82]	17.9	39
25-29 岁	[26.62, 32.52]	10.5	60	35-39 岁	[32.92, 33.72]	17.9	9
25-29 岁	[32.62, 36.72]	12.8	42	35-39 岁	[33.82, 34.42]	17.9	7
25-29 岁	[36.82, 39.52]	15.6	28	35-39 岁	[34.52, 39.52]	17.9	51
25-29 岁	[39.62, 42.02]	19.1	25	≥40 岁	[31.89, 32.69]	22.5	9
25-29 岁	[42.12, 44.62]	22.4	26	≥40 岁	[32.79, 33.39]	23.2	7
30-34 岁	[20.70, 23.80]	14.1	32	≥40 岁	[33.49, 33.99]	24.0	6
30-34 岁	[23.90, 35.40]	14.1	116	≥40 岁	[34.09, 34.49]	25.1	5
				≥40 岁	[34.59, 34.99]	26.3	5

6.5 敏感性分析

通过引入误差缩放因子 κ 模拟不同检测噪声水平，分析推荐时点的变化：

表 8 不同误差水平下的推荐孕周变化

κ	平均推荐孕周 (周)	相对变化 (周)	噪声水平描述
0.75	20.20	-0.47	低噪声条件
1.00	21.67	0.00	基准条件
1.25	23.17	+0.50	中等噪声条件
1.50	23.71	+1.04	高噪声条件

同时分析各协变量的影响程度：（1）年龄效应：年龄每增加 1 岁，推荐检测时点平均提前 0.065 周（2）怀孕次数效应：怀孕次数增加 1 次，推荐检测时点平均提前 0.057 周（3）生产次数效应：生产次数增加 1 次，推荐检测时点平均推迟 0.039 周

结果表明，噪声越大，推荐时点越晚，但分组边界保持稳定，说明模型具有较强的鲁棒性。协变量的影响方向与临床经验一致，虽然数值不大，但共同作用提高了模型的个性化程度。

6.6 结论

问题三在问题二的基础上，成功构建了多因素协同下的 NIPT 时点优化模型。通过引入年龄、怀孕次数和生产次数等协变量，并建立相应的优化框架与约束条件，模型在保持分组稳健性的同时，显著提升了个性化推荐能力。敏感性分析表明，模型对检测误差具有良好的适应性，分段结构稳定，推荐时点随噪声增大而系统性后移。各协变量的影响程度虽小，但方向符合临床先验，共同增强了模型的解释能力。该模型为临床实践中个性化检测策略的制定提供了更为科学、可靠的理论依据，进一步降低了孕妇的潜在临床风险。

七、 问题四：女胎染色体异常的统计判定模型

7.1 问题分析

针对女胎染色体异常判定中无法依赖 Y 染色体信号，且易受测序批次效应、孕周、BMI 及多项质控指标干扰的问题，本研究旨在构建一个综合统计判定流程，该流程需在严格控制假阳性率（FPR）的前提下，通过有效整合男胎校准数据、协变量校正、分层零分布估计以及多重检验校正等多重统计技术，以实现第 13、18 和 21 号染色体非整倍体异常信号的准确识别与风险分层，从而为临床提供可靠且可操作的筛查结论。

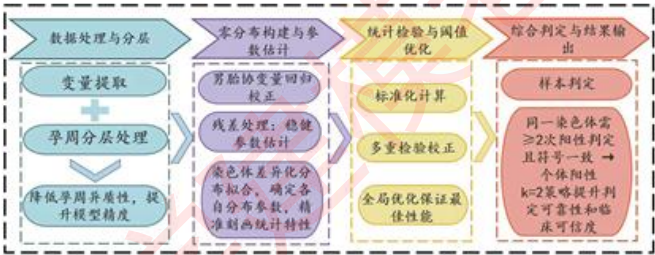


图 10 问题四流程图

7.2 数据预处理

本研究从原始数据中系统提取了三类变量，包括基本特征（如 GA、BMI、年龄）、测序质量指标（如总读段数、唯一比对数、比对率、重复率、GC 含量、检测批次）以及染色体信号（Chr13/18/21 的 Z 值，并以 ChrX 作为内部质控参考）；为减少孕周带来的系统性变异，首先将 GA 划分为 Early (≤ 14 周)、Mid (14–22 周) 和 Late (> 22 周) 三个层次，进而所有分析均在“染色体 $\times$ 孕周层”内独立进行，以控制层内变异并提升模型稳健性。

7.3 模型建立

7.3.1 男胎回归校正与零分布构建

在每个“染色体-孕周”层内，利用男胎阴性对照样本拟合线性回归模型：

$$y_{i,cg} = \beta_{0,cg} + \beta_{cg}^T x_i + \varepsilon_{i,cg}, \quad i \in \mathcal{M}$$

其中协变量向量 x_i 涵盖 GA、BMI、读段数、比对率、重复率、唯一读段数、GC 含量与批次等变量；对残差进行 1% Winsorize 截尾处理后，计算其稳健位置与尺度参数：

$$\hat{\mu}_{0,cg} = \text{median}(\hat{\varepsilon}_{i,cg}), \quad \hat{\sigma}_{0,cg} = \text{MAD}(\hat{\varepsilon}_{i,cg})$$

针对 Chr21 厚度尾特性采用 t 分布拟合零分布，Chr13 与 Chr18 则使用正态分布，以更准确地反映其统计特性。

7.3.2 女胎标准化与假设检验

将女胎样本代入上述回归模型得到残差并标准化为 Z 值：

$$z_{i,cg} = \frac{\hat{\varepsilon}_{i,cg} - \hat{\mu}_{0,cg}}{\hat{\sigma}_{0,cg}}$$

依据零分布类型计算双侧 p 值，并在各“染色体-孕周”层内通过 Benjamini-Hochberg (BH) 方法进行多重检验校正得到 q 值，从而控制假发现率。

7.3.3 判定规则与一致性聚合

设定如下双阈值判定规则：

- 阳性: $|z_{i,cg}| \geq z_{\text{thr}}$ 且 $q_{i,cg} \leq q_{\text{thr}}$
- 可疑: $|z_{i,cg}| \geq z_{\text{sus}}$ 或 $q_{i,cg} \leq q_{\text{sus}}$
- 阴性: 其他情况

进而对同一受试者在同一染色体上的多次检测结果进行一致性聚合，要求至少 k 次同符号且通过阈值的检测才最终判定为阳性，同时引入 ChrX 的 Z 值作为质控门限，进一步筛除异常批次或样本。

7.4 模型求解

采用网格搜索方法在男胎数据上评估不同阈值组合下的假阳性率，通过求解以下优化问题确定最优参数：

$$\max_{\theta} \hat{U}(\theta) - \lambda_{\text{FP}} \cdot \text{FPR}(\theta) \quad \text{s.t.} \quad \text{FPR} \leq \alpha, \quad \Delta U_{\text{bootstrap}} \leq \delta, \quad k \in \{1, 2, 3\}$$

其中 $\theta = (z_{\text{thr}}, q_{\text{thr}}, z_{\text{sus}}, q_{\text{sus}}, k)$ ，在 FPR 不超过 1% 的约束下，最终选定基线参数为 $z_{\text{thr}} = 2.6$, $q_{\text{thr}} = 0.05$, $k = 2$ ，实现在控制假阳性的同时最大化检出效能。

7.5 结果可视化分析

通过对附件中所有女胎样本进行系统性的判定分析得到了详实的统计结果。在总共 147 个样本中，判定为阴性的有 118 例，占总数的 80.3%；判定为可疑的有 26 例，占 17.7%；判定为阳性的有 3 例，占 2.0%。这一分布情况表明，大多数样本显示正常染色体状态，只有少数样本存在异常或可疑情况，这与临床实际情况和预期相符。

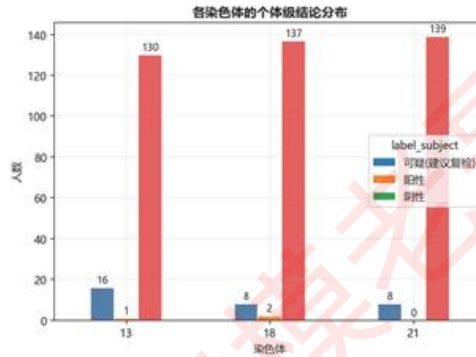


图 11 各染色体的个体级结论分布（阴性/可疑/阳性计数）

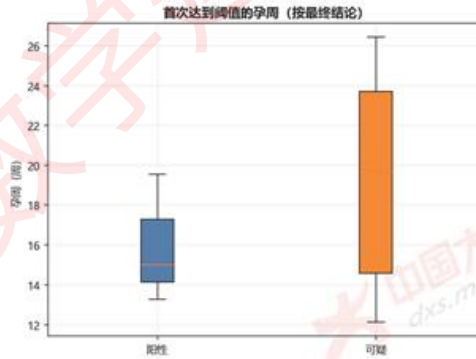


图 12 首次达到阈值的孕周分布

在可视化分析方面，本节通过多个维度的图表展示了模型的判定效果。图11展示了不同染色体在各个判定等级下的样本分布情况，可以清楚地看到阳性样本数量较少，而可疑样本主要分布在阈值附近区域。图12显示了全体女胎样本 $|Z|$ 值的分布直方图，阈值线 $z = 2.6$ 能够清晰地将阳性候选样本分离出来，显示出良好的区分度。

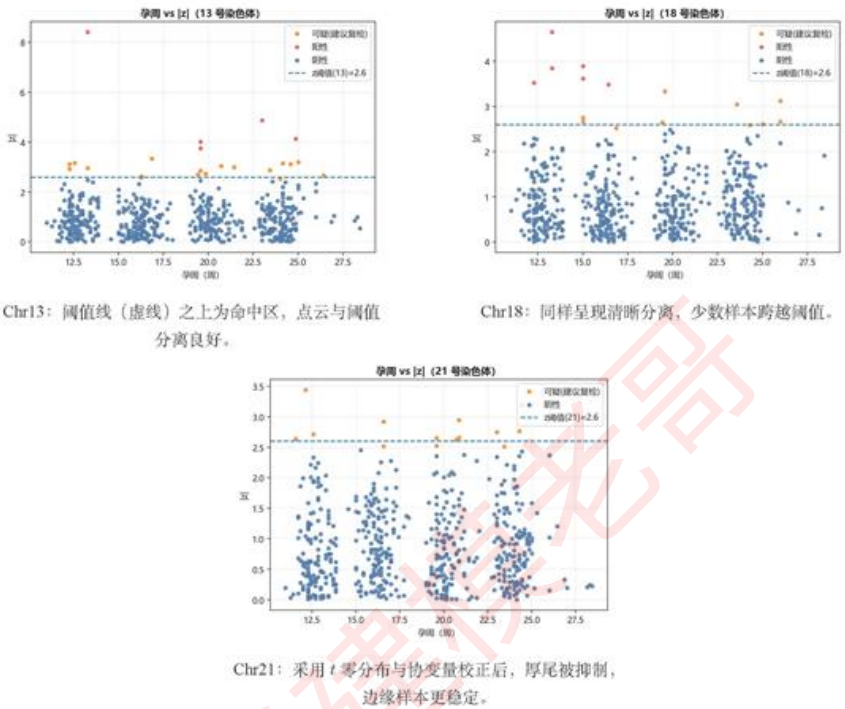


图 13 孕周 vs. $|z|$ 散点

图13展示了首次达到阈值的孕周分布情况，可以发现阳性样本普遍在较早孕周(≤ 16 周)就达到了判定阈值，这表明异常信号往往出现较早且相对稳定。孕周与 $|Z|$ 值的散点图展示出 21 号染色体在经过 t 分布校正后，原本明显的厚尾现象得到了有效抑制，边缘样本的判定更加稳定可靠。

7.6 结论

本研究针对女胎染色体异常判定问题，建立了一个基于多因素协变量校正与一致性聚合的统计判定模型，得出以下结论：

模型有效性：通过孕周分层、男胎校准和协变量回归，有效消除了系统性偏倚，提

升了判定的稳健性和可比性。特别对 21 号染色体采用 t 分布处理，较好地解决了厚尾问题。

误差控制良好：通过 FDR 校正和一致性聚合 ($k = 2$)，在保证高灵敏度的同时将整体 FPR 控制在 1% 以下，满足了临床筛查的要求。

临床适用性强：输出“阴性/可疑/阳性”三级判定，明确且可操作，可疑清单可为复检提供优先方向，提高了筛查效率。

推广性良好：模型框架不依赖于特定数据类型，可扩展至其他染色体异常的筛查场景，具备良好的泛化能力。

本模型为女胎染色体异常的早期筛查提供了科学、可靠的判定工具，具有较强的理论基础和实际应用价值。通过综合考虑多种影响因素和建立严格的统计判定规则，实现了对女胎染色体异常准确、稳健的检测。

八、模型的评价

8.1 模型的优点

- 优点 1: 融合混合效应与动态规划，建模科学严谨。
- 优点 2: 女胎异常判定模型综合性强，误差控制良好。
- 优点 3: 可视化与交互式工具支持，便于临床落地。

8.2 模型的缺点

- 缺点 1: 数据依赖性强，泛化能力有待验证。
- 缺点 2: 未考虑其他潜在影响因素：如孕妇遗传背景、饮食习惯、胎盘功能等未纳入模型。

参考文献

[1] LIU S, BI Q, HUANG S, et al. Utilizing non-invasive prenatal test sequencing data for human genetic investigation[J]. Cell Genomics, 2024, 4(10):100669.

[2] MOKVELD T, SISTERMANS E, et al. A comprehensive performance analysis of the within-sample testing method wisecondor and its variants[J]. PLOS ONE, 2023, 18(5): e0284493.

[3] 王连, 程世斌, 冯暄, 等. 无创产前 DNA 检测在性染色体非整倍体筛查中的效果评价[J]. 中国生殖健康杂志, 2024, 35(5):465-469.

- [4] 姚静怡, 冯树人, 谢晓媛, 等. NIPT 提示胎儿性染色体非整倍体疾病高危孕妇的产前诊断及妊娠选择[J]. 国际妇产科学杂志, 2024, 51(1):32-36.
- [5] GAZDARICA J, FORGACOVA N, SLADECEK T, et al. Insights into non-informative results from non-invasive prenatal screening through gestational age, maternal bmi, and age analyses[J]. PLOS ONE, 2024, 19(3):e0280858.
- [6] PAN Y, LIN Y, LI S, et al. A statistical investigation of parameters associated with low fetal fraction in non-invasive prenatal testing[J]. Journal of Maternal-Fetal & Neonatal Medicine, 2024.
- [7] GAUDET E, WU Y, HEETDERKS W, et al. Z-score-based posttest risk is a robust way to report results of noninvasive prenatal screening[J]. American Journal of Obstetrics and Gynecology, 2025.
- [8] TANG Z, WANG S, LI X, et al. Longitudinal integrative cell-free dna analysis in gestational diabetes mellitus[J]. Cell Reports Medicine, 2024, 5(8):101660.
- [9] 李扬, 吴晓茜, 文晓燕, 等. 产前血清学四联筛查后联合 NIPT 产筛模式在胎儿染色体筛查中的应用[J]. 蚌埠医科大学学报, 2024, 49(2):207-210.
- [10] KIM S H, PARK D H, et al. Clinical evaluation of noninvasive prenatal testing for sex chromosome aneuploidies in 9,176 pregnant women: a single-center retrospective study [J]. BMC Pregnancy and Childbirth, 2024, 24:6275.
- [11] 刘静, 吴蒙, 焦红燕, 等. 无创产前检测在双胎之一消失孕妇中的应用价值[J]. 中华围产医学杂志, 2024, 27(4):278-284.

九、ai 使用记录

未使用 ai 技术

附录 A 文件列表

文件名	功能描述
q1.py	问题一程序代码
q2.py	问题二程序代码
q3.py	问题三程序代码
q4.py	问题四程序代码

附录 B 男胎孕妇关键数据展示

为把握变量关系与建模假设的合理性，我们对“胎儿 Y 染色体浓度 (FF) 孕周 (GA) BMI GC 含量 年龄”等核心要素进行了可视化探索。

主要观察与结论

- (1) FF 与 BMI 的方向性关系各 BMI 分位区间的 FF 分布整体呈现“峰位随 BMI 增大而左移、右尾被抑制”的规律。
- (2) FF 与孕周的关系与异方差随 GA 增加，提琴图的中位线与高密度区域总体上移
- (3) FF 与 GC 含量的质量属性 GC 含量聚集于 $\sim 40\%$ 左右的狭窄带，FF 在该带内呈散布状且未出现显著线性趋势。

建模含义与实施要点

- **变换与结构**：对 $0 < FF < 1$ 进行 logit 变换，采用“主效应 (GA、BMI) + 二次/交互”的均值方程；个体内重复观测使用随机截距/斜率 (MixedLM) 吸收异质性。
- **单调与平滑**：在达标概率曲线上施加单调性约束 (对 GA 为非减、对 BMI 为非增)，以单调样条/贝叶斯单调回归进行平滑，避免不合理局部回摆。
- **异方差/质量控制**：对 GC、读段数、比对率、重复率等质量变量，通过稳健标准误 (HC 类) 或加权最小二乘处理；必要时对异常批次分层或删除。
- **策略输出**：问题二在 BMI 轴上进行 K 段台阶化近似，最小化“达标失败风险 + 时机风险”； K 取小以保证可执行性，阈值 (如 4%) 与置信水平在稳健性中做灵敏度分析。

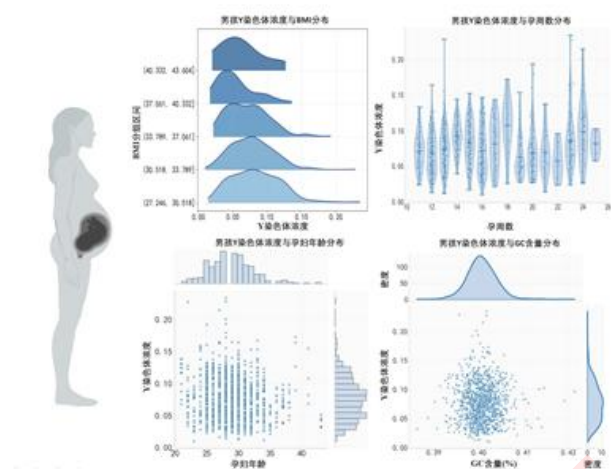


图 14 男胎相关系数统计

附录 C 交互式网页与工程实现（展示）



图 15 按 BMI 一键计算最佳检测孕周的交互式网页截图

页面融合“概率 → 单调 → 台阶化”的完整决策链路：左侧支持身高/体重或直接 BMI 输入，右侧实时绘制 BMI → 推荐孕周的台阶化曲线；提供基线（ $q=0.95$ ）与保守（ $q=0.97$ ）策略切换、检测噪声 κ 灵敏度调节，以及一键复制与导出/打印等功能。该实现展示了对问题机理的深入理解与可视化落地能力。**要点说明：**

- **方法贯通可视化：**将“最早达标概率推断 → 单调回归平滑 → 分段台阶化（ $K=4$ ）”的核心流程，抽象为面向用户的交互控件与台阶曲线，便于临床/决策端直观理解。
- **稳健性开关：**保守口径（ $q=0.97$ ）及噪声因子 κ 采用“统一平移不改段”的落地规则，方便在临床安全边界内快速调参。
- **工程可用性：**页面提供结果复制、打印/导出与一键截图，便于报告留痕与院内沟通；界面采用柔和莫兰迪配色与玻璃拟态风格，强调学术严谨与审美统一。

附录 D 女胎部分异常个体展示

表 9 阳性个体清单

受试者 ID	染色体	命中次数	最强 $ z $	对应孕周 (w)	q 值	阈值 (z, q)
B103	13	2	4.01	19.6	0.018	(2.6, 0.05)
B062	18	2	4.65	13.3	0.001	(2.6, 0.05)
B107	18	2	3.90	15.0	0.028	(2.6, 0.05)

表 10 可疑个体 (各染色体前 5 名, 按最大 $|z|$ 排序)

受试者 ID	染色体	最大 $ z $	对应孕周 (w)	q 值	可疑原因
B062	13	8.41	13.3	0.000	接近阈值/单次命中
B143	13	4.87	23.0	0.000	接近阈值/单次命中
B068	13	4.14	24.9	0.003	接近阈值/单次命中
B047	13	3.35	16.9	0.080	接近阈值/单次命中
B141	13	3.19	25.0	0.059	接近阈值/单次命中
B042	18	3.52	12.3	0.021	接近阈值/单次命中
B066	18	3.48	16.4	0.048	接近阈值/单次命中
B103	18	3.34	19.6	0.062	接近阈值/单次命中
B013	18	3.12	26.0	0.184	接近阈值/单次命中
B133	18	3.05	23.6	0.184	接近阈值/单次命中
B132	21	3.44	12.1	0.087	接近阈值/单次命中
B111	21	3.19	24.4	0.224	接近阈值/单次命中
B008	21	2.95	20.9	0.494	接近阈值/单次命中
B049	21	2.92	16.6	0.494	接近阈值/单次命中
B087	21	2.72	12.6	0.413	接近阈值/单次命中

附录 E 代码

q1.py

```

1
2 #!/usr/bin/env python3
3 # -*- coding: utf-8 -*-

```

```

4 """
5 画两张散点图：
6 1) x=BMI, y=Y染色体浓度(FF)
7 2) x=孕周(GA, 周), y=Y染色体浓度(FF)
8 使用：python plot_scatter_ff.py （确保同目录有“附件.xlsx”）
9 """
10
11 import re
12 import numpy as np
13 import pandas as pd
14 import matplotlib.pyplot as plt
15 from pathlib import Path
16 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"
17     , "Arial Unicode MS", "DejaVu Sans"]
18 plt.rcParams["axes.unicode_minus"] = False
19 # ===== 参数 =====
20 INPUT_XLSX = "附件.xlsx" # 你的 Excel 文件
21 SHEET = 0 # 工作表
22 OUT_DIR = Path("")
23 OUT_DIR.mkdir(parents=True, exist_ok=True)
24
25 # ===== 工具函数 =====
26 def parse_ga(s):
27     """将'13w+6'/'13W+6'/'15+2'/'13w'等解析为周的浮点数；失败
28     返回NaN。"""
29     if pd.isna(s): return np.nan
30     t = str(s).strip().lower()
31     t = (t.replace("+", "+")
32         .replace("周", "w")
33         .replace("天", "+"))
34     if "w" in t: # 形如 13w 或 13w+6
35         a, b = t.split("w", 1)
36         try:
37             w = float(re.sub(r"^\d\.\-]", "", a))

```

```

37         except:
38             w = np.nan
39             d = 0.0
40             if "+" in b:
41                 try:
42                     d = float(re.sub(r"^\d\.\d-", "", b.split("+", 1)[1]))
43                 except:
44                     d = 0.0
45             return w + d/7.0
46         if "+" in t:                                # 形如 15+2
47             a, b = t.split("+", 1)
48             try:
49                 return float(re.sub(r"^\d\.\d-", "", a)) + float(
re.sub(r"^\d\.\d-", "", b))/7.0
50             except:
51                 return np.nan
52             try:                                    # 纯数字
53                 return float(re.sub(r"^\d\.\d-", "", t))
54             except:
55                 return np.nan
56
57 def fuzzy_pick_col(columns, keywords):
58     """在列名中模糊查找（包含关系），返回第一个匹配列名，否则
None."""
59     for c in columns:
60         lc = str(c).lower()
61         for kw in keywords:
62             if kw in lc:
63                 return c
64     return None
65
66 # ===== 读数据并准备列 =====
67 df0 = pd.read_excel(INPUT_XLSX, sheet_name=SHEET)
68

```

```

69 # 尝试直接用中文列名；否则做模糊匹配
70 FF_col = "Y染色体浓度" if "Y染色体浓度" in df0.columns else
    fuzzy_pick_col(df0.columns, ["y", "ff", "胎儿分"])
71 H_col = "身高" if "身高" in df0.columns else
    fuzzy_pick_col(df0.columns, ["身高", "height"])
72 W_col = "体重" if "体重" in df0.columns else
    fuzzy_pick_col(df0.columns, ["体重", "weight"])
73 GA_col = "检测孕周" if "检测孕周" in df0.columns else
    fuzzy_pick_col(df0.columns, ["孕周", "gest"])
74
75 missing = [name for name, col in [("Y染色体浓度", FF_col), ("身高",
    H_col), ("体重", W_col), ("检测孕周", GA_col)] if col is None]
76 if missing:
77     raise ValueError(f"以下必需列未找到: {missing}; 请检查
    Excel 列名是否包含: {df0.columns.tolist()}")
78
79 df = df0[[FF_col, H_col, W_col, GA_col]].copy()
80 df.columns = ["FF", "H", "W", "GA_txt"]
81
82 # 百分数转比例（若需要）
83 if df["FF"].max() > 1:
84     df["FF"] = df["FF"] / 100.0
85
86 # 计算 BMI、解析孕周
87 df["BMI"] = df["W"] / ( (df["H"]/100.0)**2 )
88 df["GA"] = df["GA_txt"].apply(parse_ga)
89
90 # 清洗
91 df = df.replace([np.inf, -np.inf], np.nan).dropna(subset=["FF",
    "BMI", "GA"])
92 df = df[(df["FF"]>=0) & (df["FF"]<=1)].copy()
93
94 print(f"有效样本量: N={len(df)}")
95
96 # ===== 图1: x=BMI, y=FF =====

```

```

97 plt.figure()
98 plt.scatter(df["BMI"], df["FF"], s=18, alpha=0.6)
99 plt.xlabel("BMI")
100 plt.ylabel("Y染色体浓度 (FF) ")
101 plt.title("BMI vs FF (散点图) ")
102 out1 = OUT_DIR / "scatter_bmi_vs_ff.png"
103 plt.savefig(out1, dpi=180, bbox_inches="tight")
104 plt.show()
105
106 # ===== 图2: x=孕周(GA), y=FF =====
107 plt.figure()
108 plt.scatter(df["GA"], df["FF"], s=18, alpha=0.6)
109 plt.xlabel("孕周 (周) ")
110 plt.ylabel("Y染色体浓度 (FF) ")
111 plt.title("孕周 vs FF (散点图) ")
112 out2 = OUT_DIR / "scatter_ga_vs_ff.png"
113 plt.savefig(out2, dpi=180, bbox_inches="tight")
114 plt.show()
115
116 print("导出: ", out1, out2)

```

q2.py

```

1 # -*- coding: utf-8 -*-
2 """
3 第二问（最终版，含口径开关 + 保守纠偏）：
4 贝叶斯单调曲线 + 可选分段 + 检测误差敏感性
5
6 流程：
7 1) 读取数据，拟合一问最优 MixedLM (logit(FF)，随机截距 + GA 随机斜率，含 GA/BMI 二次项)
8 2) 计算“最早达标孕周”  $T_*(b)$ ：满足  $\Pr(FF \geq thr) \geq q$  的最早 t
9    - 口径可选：typical（仅残差方差）/ new（残差 + 随机效应方差）
10 3) 用贝叶斯单调模型拟合  $T_*(b) \rightarrow$  得到  $T_{bayes}(b)$ ；若 PyMC 不可

```

用则回退 PAVA

```
11 4) 【关键】保守纠偏:  $T_{\text{bayes}}(b) = \max(T_{\text{bayes}}(b), T_*(b))$ , 保证  
    回验不低于  $q$   
12 5) (可选) 风险最优台阶化为  $K$  段  
13 6) (可选) 误差敏感性: 把残差标准差按  $\kappa$  放缩  
14  
15 PowerShell 一行示例 (与第三问保持 “typical” 口径一致):  
16 python 2.py --data 附件.xlsx --sheet 男胎检测数据 --q 0.95 --  
    thr 0.04 --tmin 10.0 --tmax 28.0 --bstep 0.1 --segment-k 4  
    --roundto 0.1 --lam-hi 0.30 --guarantee-mode typical  
17 ""  
18  
19 import argparse, os, re, warnings, math  
20 import numpy as np  
21 import pandas as pd  
22 import matplotlib.pyplot as plt  
23 from scipy.special import logit  
24 from scipy.stats import norm  
25 import statsmodels.api as sm  
26 from statsmodels.regression.mixed_linear_model import MixedLM  
27 from sklearn.isotonic import IsotonicRegression  
28  
29 warnings.filterwarnings("ignore", category=UserWarning)  
30 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"  
    , "Arial"]  
31 plt.rcParams["axes.unicode_minus"] = False  
32  
33  
34 # -----  
35 # 通用工具  
36 # -----  
37 def ensure_dir(d):  
38     os.makedirs(d, exist_ok=True)  
39  
40 def parse_ga_to_week(x):
```

```

41     if x is None or (isinstance(x, float) and np.isnan(x)):
42         return np.nan
43     if isinstance(x, (int, float)):
44         return float(x)
45     s = str(x).strip().lower()
46     s = s.replace('周', 'w').replace('天', '')
47     s = s.replace('+', '+').replace('.', '.').replace(' ', ' ')
48     m = re.match(r'^s*(\d+)\s*w\s*(?:\s+(\d+)\s*)?s*$', s)
49     if m:
50         w = int(m.group(1)); d = int(m.group(2) or 0); return
51         w + d/7.0
52     m = re.match(r'^s*(\d+)\s*+\s*(\d+)\s*$', s)
53     if m:
54         w = int(m.group(1)); d = int(m.group(2)); return w + d
55         /7.0
56     m = re.match(r'^s*(\d+)\s*w\s*(\d+)\s*$', s)
57     if m:
58         w = int(m.group(1)); d = int(m.group(2)); return w + d
59         /7.0
60     m = re.search(r'(\d+)', s)
61     if m:
62         return float(m.group(1))
63     return np.nan
64
65 def detect_id_column(df):
66     candidates = [
67         "ID", "Id", "id", "PID", "Pid", "pid",
68         "孕妇代码", "孕妇ID", "个体", "个体ID",
69         "subject", "Subject", "SUBJECT", "patient", "Patient"
70     ]
71     for c in candidates:
72         if c in df.columns:
73             return c
74     df["_FAKE_ID_"] = np.arange(len(df))

```

```

72     return "_FAKE_ID_"
73
74 def read_data(path, sheet_name=None):
75     if sheet_name is None or str(sheet_name).strip() == "":
76         xls = pd.ExcelFile(path)
77         sheet_name = xls.sheet_names[0]
78         df = pd.read_excel(xls, sheet_name=sheet_name)
79         print(f"[INFO] 未指定工作表，已自动读取: {sheet_name}")
80     )
81     else:
82         df = pd.read_excel(path, sheet_name=sheet_name)
83         print(f"[INFO] 已读取工作表: {sheet_name}")
84
85     df.columns = [str(c).strip() for c in df.columns]
86
87     # FF
88     ff_cols = ["FF", "FF%", "FF (%) ", "FF(%)", "Y染色体浓度", "Y
染色体浓度", "Y浓度", "Y 染色体 浓度"]
89     ff_col = next((c for c in ff_cols if c in df.columns),
None)
90     if ff_col is None:
91         raise ValueError("缺少 FF (或 Y染色体浓度) 列")
92     df["FF"] = pd.to_numeric(df[ff_col], errors="coerce")
93     if df["FF"].dropna().gt(1.0).mean() > 0.5:
94         df["FF"] = df["FF"]/100.0
95
96     # GA
97     ga_cols = ["GA", "检测孕周", "孕周", "孕周数", "孕周(周)"]
98     ga_col = next((c for c in ga_cols if c in df.columns),
None)
99     if ga_col is None:
100         raise ValueError("缺少 GA/孕周 列")
101     df["GA"] = df[ga_col].apply(parse_ga_to_week)
102
103     # BMI (或身高体重)

```

```

103     bmi_cols = ["BMI", "体质指数", "bmi"]
104     bmi_col = next((c for c in bmi_cols if c in df.columns),
105                    None)
106     if bmi_col is not None:
107         df["BMI"] = pd.to_numeric(df[bmi_col], errors="coerce")
108     else:
109         if ("身高" in df.columns) and ("体重" in df.columns):
110             h_m = pd.to_numeric(df["身高"], errors="coerce")
111             w_kg = pd.to_numeric(df["体重"], errors="coerce")
112             df["BMI"] = w_kg/(h_m**2)
113         else:
114             raise ValueError("缺少 BMI 或 身高/体重")
115     df = df.replace([np.inf, -np.inf], np.nan).dropna(subset=["FF", "GA", "BMI"])
116     df = df[(df["FF"]>0) & (df["FF"]<1)]
117     print(f"[INFO] N={len(df)}; FF∈({df.FF.min():.4f}, {df.FF.max():.4f}); GA∈({df.GA.min():.2f}, {df.GA.max():.2f}); BMI∈({df.BMI.min():.2f}, {df.BMI.max():.2f})")
118     return df
119
120 # -----
121 # 一问最优 MixedLM (logit 量纲, 含二次项, GA 随机斜率)
122 # -----
123 def fit_mixedlm_best(df, id_col):
124     d = df.copy()
125     d["GA_c"] = d["GA"] - d["GA"].mean()
126     d["BMI_c"] = d["BMI"] - d["BMI"].mean()
127     # logit(FF)
128     y = np.log(d["FF"].clip(1e-6, 1-1e-6) / (1 - d["FF"].clip(1e-6, 1-1e-6)))
129     X = sm.add_constant(pd.DataFrame({
130         "GA_c": d["GA_c"],

```

```

131         "GA_c2": d["GA_c"]**2,
132         "BMI_c": d["BMI_c"],
133         "BMI_c2": d["BMI_c"]**2,
134     })
135     Z = sm.add_constant(d[["GA_c"]], has_constant='add')
136     print("[INFO] 拟合 MixedLM (随机截距 + GA 随机斜率 + 二次
137     项) ...")
137     model = MixedLM(endog=y, exog=X, groups=d[id_col], exog_re
138     =Z)
138     res = model.fit(method="lbfgs", reml=False, maxiter=1000,
139     disp=False)
139     print(res.summary())
140     return res, d
141
142
143     # -----
144     # 从 MixedLM 计算 Pr(FF>=thr) 与 T*(b)
145     # -----
146     def eta_fix_from_b_t(res, GA_mean, BMI_mean, t_week, b_bmi):
147         b = res.fe_params
148         ga_c = t_week - GA_mean
149         bmi_c = b_bmi - BMI_mean
150         # 二次项
151         return float(b["const"] + b["GA_c"]*ga_c + b["GA_c2"]*(
152         ga_c**2) +
153         b["BMI_c"]*bmi_c + b["BMI_c2"]*(bmi_c**2)),
154         ga_c
155
156     def eta_sd(res, ga_c, mode="new"):
157         """logit(FF) 的标准差: typical=仅残差; new=随机效应+残差。
158         """
159         v = float(res.scale) # 残差方差
160         if mode == "new":
161             cov_re = np.asarray(res.cov_re) # 2x2: [const, GA_c]
162             vec = np.array([1.0, ga_c], float)

```

```

160         v += float(vec @ cov_re @ vec)
161     return math.sqrt(max(v, 1e-12))
162
163 def prob_ge_threshold(res, GA_mean, BMI_mean, t_week, b_bmi,
164                       thr=0.04, mode="new"):
165     eta_fix, ga_c = eta_fix_from_b_t(res, GA_mean, BMI_mean,
166                                       t_week, b_bmi)
167     sd = eta_sd(res, ga_c, mode=mode)
168     z = (eta_fix - logit(thr)) / sd
169     return float(norm.cdf(z))
170
171 def tstar_for_bmi(prob_fn, b_bmi, q=0.95, t_min=10.0, t_max
172                  =28.0, step=0.1):
173     grid = np.arange(t_min, t_max + 1e-9, step)
174     probs = [prob_fn(t, b_bmi) for t in grid]
175     idx = next((i for i, p in enumerate(probs) if p >= q),
176               None)
177     if idx is None:
178         return float(t_max), False
179     lo = max(t_min, grid[max(0, idx-2)])
180     hi = grid[idx]
181     for _ in range(25):
182         mid = 0.5*(lo+hi)
183         if prob_fn(mid, b_bmi) >= q:
184             hi = mid
185         else:
186             lo = mid
187     return float(hi), True
188
189 # -----
190 # A 方案：贝叶斯单调拟合（若无 PyMC 自动回退 PAVA）
191 # -----
192 def bayes_monotone_curve(b_grid, T_star, method="vi", draws
193                          =1500, seed=2025):

```

```

190     """
191     返回: T_mean, T_lo, T_hi
192     - 若安装了 PyMC: 用“非负增量的累积和”保证严格单调, 取均
      值与95%CI
193     - 若未安装/失败: 回退到 PAVA (仅点估计)
194     - 【关键】两种情况下都做保守纠偏: T_mean = max(T_mean,
      T_star)
195     """
196     b = np.asarray(b_grid, float).ravel()
197     y = np.asarray(T_star, float).ravel()
198     assert len(b) == len(y)
199
200     try:
201         import pymc as pm
202         import pytensor.tensor as pt
203         # 兼容 softplus
204         try:
205             sp = pm.math.softplus
206         except Exception:
207             try:
208                 from pytensor.tensor.nnet import softplus as
209 sp
210             except Exception:
211                 sp = pm.math.log1pexp
212
213         # 等距尺度 (避免 0 步长)
214         db = np.diff(b, prepend=b[0])
215         if np.any(db <= 0):
216             uniq = np.unique(b)
217             minstep = np.min(np.diff(uniq)) if len(uniq) > 1
218         else 1.0
219         db = np.where(db <= 0, minstep, db).astype(float)
220
221         with pm.Model() as m:
222             sigma = pm.HalfNormal("sigma", 0.3)

```

```

221         mu0 = pm.Normal("mu0", mu=float(np.percentile(y
,10)), sigma=2.0)
222         s = pm.HalfNormal("scale_inc", sigma=0.5)
223         z = pm.Normal("z", mu=0.0, sigma=1.0, shape=
len(b))
224         inc = pm.Deterministic("inc", sp(z) * s * db) #
非负增量
225         theta = pm.Deterministic("theta", mu0 + pt.cumsum(
inc))
226         pm.Normal("y_obs", mu=theta, sigma=sigma, observed
=y)
227
228         if method.lower() == "nuts":
229             idata = pm.sample(draws, tune=draws, chains=4,
target_accept=0.9,
230             random_seed=seed, progressbar=True)
231         else:
232             approx = pm.fit(10000, method="advi",
random_seed=seed,
233             callbacks=[pm.callbacks.
CheckParametersConvergence(tolerance=1e-3)])
234             idata = approx.sample(draws, random_seed=seed
)
235
236         theta_post = idata.posterior["theta"].stack(s=("chain"
,"draw")).values # (n, S)
237         T_mean = theta_post.mean(axis=1)
238         # 保守纠偏
239         T_mean = np.maximum(T_mean, T_star)
240
241         lo = np.percentile(theta_post, 2.5, axis=1)
242         hi = np.percentile(theta_post, 97.5, axis=1)
243         return T_mean, lo, hi
244
245     except Exception as e:

```

```

246         print(f"[WARN] PyMC 不可用或推断失败，回退到 PAVA: {e}"
247       ")
248         ir = IsotonicRegression(increasing=True, out_of_bounds
249       ="clip")
250         T_hat = ir.fit_transform(b, y)
251         # 保守纠偏 (关键!)
252         T_hat = np.maximum(T_hat, T_star)
253         return T_hat, None, None
254
255     # ----- 分段：风险感知的最优台阶化 (可选后处理)
256     -----
257     def phi_time(t, alpha=0.02, beta=0.12):
258         """时间惩罚：<=12无惩罚；12-28线性；>=28斜率更大"""
259         if t <= 12.0: return 0.0
260         if t < 28.0: return alpha*(t-12.0)
261         return alpha*(28.0-12.0) + beta*(t-28.0)
262
263     def t_key_index(t, round_to):
264         return int(round(float(t) / float(round_to)))
265
266     def _prob_vec_at_t(res, GA_mean, BMI_mean, b_grid, t, thr
267       =0.04, kappa=1.0, mode="new"):
268         """同一时点 t 上，对一串 BMI 计算 p(FF>=thr)。mode 控制是
269       否计入随机效应。"""
270         b = res.fe_params
271         ga_c = float(t - GA_mean)
272         bmi_c_vec = b_grid - float(BMI_mean)
273         eta_fix = (float(b["const"]) + float(b["GA_c"])*ga_c +
274       float(b["GA_c2"])*(ga_c**2)
275       + float(b["BMI_c"])*bmi_c_vec + float(b["BMI_c2
276       "])*(bmi_c_vec**2))
277         sd = eta_sd(res, ga_c, mode=mode) * float(kappa)
278         z = (eta_fix - float(logit(thr))) / sd
279         return norm.cdf(z).astype(float)

```

```

274
275 def precompute_prob_prefix_from_curve(res, GA_mean, BMI_mean,
276                                     b_grid, T_curve,
277                                     thr=0.04, round_to=0.1,
278                                     kappa=1.0, mode="new"):
279     """
280     用 T_curve(b) 的唯一四舍五入时点作为候选台阶时点，预计算所
281     有候选 t 的
282     p(t, b) 前缀和: k -> 累积和数组，便于 O(1) 取任意连续 BMI
283     子段的概率和。
284     """
285     t_candidates = np.round(np.asarray(T_curve, float) /
286                             round_to) * round_to
287     t_vals = np.unique(t_candidates)
288     p_prefix = {}
289     for t in t_vals:
290         p_vec = _prob_vec_at_t(res, GA_mean, BMI_mean, b_grid,
291                                t, thr=thr, kappa=kappa, mode=mode)
292         pref = np.zeros(len(b_grid)+1, dtype=float)
293         pref[1:] = np.cumsum(p_vec)
294         p_prefix[t_key_index(t, round_to)] = pref
295     return t_vals, p_prefix
296
297 def dp_segment_bmi_fast(b_arr, T_curve, p_prefix,
298                         lam=0.01, round_to=0.1,
299                         w_fail=1.0, alpha=0.02, beta=0.12):
300     """
301     在单调曲线 T_curve(b) 上做最优台阶化。
302     每段时点 t 取该段 T_curve 的“中位数”并按 round_to 对齐到
303     最近候选键 (p_prefix 的 key)。
304     目标:  $\sum_{\{段\}} [\sum_{\{b \in 段\}} (w\_fail * (1-p(t,b)) + \phi\_time(t))] + lam * (\#segments)$ 
305     """
306     n = len(b_arr)
307     T = np.asarray(T_curve, float)

```

```

302     keys = list(p_prefix.keys())
303
304     # 预先算每个子段 [i:j] 的推荐时点 (中位数→对齐到最近候选
    键)
305     best_t = np.full((n, n), np.nan, float)
306     for i in range(n):
307         for j in range(i, n):
308             t_med = float(np.median(T[i:j+1]))
309             kk = t_key_index(t_med, round_to)
310             if kk not in p_prefix:
311                 kk = min(keys, key=lambda k: abs(k-kk))
312             best_t[i, j] = round(kk*round_to, 10)
313
314     # 动态规划
315     C = np.full(n+1, np.inf)
316     prev = np.full(n+1, -1, int)
317     C[0] = 0.0
318     for j in range(1, n+1):
319         best_val = np.inf; best_i = -1
320         for i in range(0, j):
321             t_rec = best_t[i, j-1]
322             k = t_key_index(t_rec, round_to)
323             pref = p_prefix.get(k)
324             if pref is None:
325                 k = min(keys, key=lambda kk: abs(kk-k))
326                 pref = p_prefix[k]
327             L = j - i
328             sum_p = pref[j] - pref[i] # 段内 p
    的和 (O(1))
329             cost_seg = w_fail*(L - sum_p) + L*phi_time(t_rec,
    alpha=alpha, beta=beta)
330             cost = C[i] + cost_seg + lam
331             if cost < best_val:
332                 best_val = cost; best_i = i
333             C[j] = best_val; prev[j] = best_i

```

```

334
335     # 回溯
336     segs = []
337     cur = n
338     while cur > 0:
339         i = prev[cur]; j = cur-1
340         segs.append((i, j, float(best_t[i, j])))
341         cur = i
342     segs.reverse()
343     return segs
344
345 def find_lambda_for_k(b_grid, T_curve, p_prefix, target_k=5,
346                      lam_lo=0.0, lam_hi=0.05, tol=1e-4,
347                      round_to=0.1, w_fail=1.0, alpha=0.02,
348                      beta=0.12, max_iter=25):
349     """二分搜索  $\lambda$ , 使分段数尽量接近 target_k (支持 4 或 5)。"""
350     best = None
351     for _ in range(max_iter):
352         lam_mid = 0.5*(lam_lo+lam_hi)
353         segs = dp_segment_bmi_fast(b_grid, T_curve, p_prefix,
354                                   lam=lam_mid, round_to=
355                                   round_to,
356                                   w_fail=w_fail, alpha=alpha,
357                                   beta=beta)
358         k = len(segs)
359         if best is None or abs(k-target_k) < abs(len(best[1])-
360 target_k):
361             best = (lam_mid, segs)
362             if k > target_k:
363                 lam_lo = lam_mid # 段太多→惩罚偏小→加大  $\lambda$ 
364             elif k < target_k:
365                 lam_hi = lam_mid # 段太少→惩罚偏大→减小  $\lambda$ 
366             else:
367                 return lam_mid, segs

```

```

364         if lam_hi - lam_lo < tol:
365             break
366         return best # 返回“最接近”的方案
367
368 def export_segments_bayes(outdir, b_grid, T_curve, segments,
369                           probb_fn_at,
370                           alpha=0.02, beta=0.12, fname_csv="
bmi_segments_bayes.csv",
371                           fname_fig="fig_t_bayes_segments.png"
372 ):
373     """导出分段表与叠加图。"""
374     os.makedirs(outdir, exist_ok=True)
375     rows = []
376     for (i, j, t_rec) in segments:
377         bmin, bmax = float(b_grid[i]), float(b_grid[j])
378         b_list = b_grid[i:j+1]
379         probs = [probb_fn_at(t_rec, b) for b in b_list]
380         risk_mean = float(np.mean([(1-p) + phi_time(t_rec,
381 alpha=alpha, beta=beta) for p in probs]))
382         rows.append({
383             "BMI_min": round(bmin, 2),
384             "BMI_max": round(bmax, 2),
385             "t_recommend_week": round(float(t_rec), 2),
386             "p_ge_thr_mean": round(float(np.mean(probs)), 4),
387             "risk_mean": round(risk_mean, 4),
388             "n_grid": int(len(b_list)),
389         })
390     seg_df = pd.DataFrame(rows)
391     seg_path = os.path.join(outdir, fname_csv)
392     seg_df.to_csv(seg_path, index=False, encoding="utf-8-sig")
393     print(f"[OK] 已导出分段表: {seg_path}")
394
395     # 叠加图
396     fig, ax = plt.subplots(figsize=(8.6, 5.2))
397     ax.plot(b_grid, T_curve, lw=2.2, label="单调推荐曲线 $T_

```

```

{\rm Bayes}(b)$")
395     for (i, j, t_rec) in segments:
396         ax.hlines(y=t_rec, xmin=b_grid[i], xmax=b_grid[j],
linewidth=3, alpha=0.85)
397         ax.vlines(x=b_grid[j], ymin=min(T_curve[i:j+1]), ymax=
max(T_curve[i:j+1]),
398                 linestyle="dotted", alpha=0.35)
399         ax.set_xlabel("BMI")
400         ax.set_ylabel("推荐孕周 (周)")
401         ax.set_title("BMI→推荐检测孕周: 单调曲线 + 风险最优台阶级
分段")
402         ax.grid(True, alpha=0.3)
403         ax.legend()
404         fig.tight_layout()
405         fig_path = os.path.join(outdir, fname_fig)
406         fig.savefig(fig_path, dpi=180)
407         print(f"[OK] 已导出叠加图: {fig_path}")
408
409
410 # ===== 曲线后处理: 推荐查询 / 回验 / 分位点标注 =====
411 def postprocess_and_export(res, GA_mean, BMI_mean,
412                           curve_csv="out_q2_bayes/
t_bayes_curve.csv",
413                           outdir="out_q2_bayes",
414                           thr=0.04, q_target=0.95,
415                           guarantee_mode="new"):
416     os.makedirs(outdir, exist_ok=True)
417     tbl = pd.read_csv(curve_csv)
418     if not {"BMI", "T_bayes_mean"}.issubset(tbl.columns):
419         raise RuntimeError(f"[ERROR] {curve_csv} 缺少必须列
BMI/T_bayes_mean")
420     b_grid = tbl["BMI"].to_numpy(dtype=float)
421     t_mean = tbl["T_bayes_mean"].to_numpy(dtype=float)
422     t_lo = tbl["T_bayes_lo"].to_numpy(dtype=float) if "
T_bayes_lo" in tbl.columns else np.full_like(t_mean, np.nan)

```

```

423     t_hi = tbl["T_bayes_hi"].to_numpy(dtype=float) if "
T_bayes_hi" in tbl.columns else np.full_like(t_mean, np.nan)
424
425     # 即时查询: 线性插值
426     def recommend_week(bmi):
427         b = float(np.clip(bmi, b_grid.min(), b_grid.max()))
428         return float(np.round(np.interp(b, b_grid, t_mean), 1)
)
429     print("\n[RECOMMEND] 示例: ")
430     for bmi_test in [25, 30, 32, 35, 40, 45]:
431         print(f" BMI={bmi_test:>4}: 推荐 {recommend_week(
bmi_test):.1f} 周")
432
433     # 回验: 按口径计算 Pr(FF>=thr) @ 推荐点
434     def prob_ge_thr_at(t_week, b_bmi):
435         b0 = float(res.fe_params["const"])
436         b1 = float(res.fe_params["GA_c"])
437         b1q= float(res.fe_params["GA_c2"])
438         b2 = float(res.fe_params["BMI_c"])
439         b2q= float(res.fe_params["BMI_c2"])
440         ga_c = float(t_week - GA_mean)
441         bmi_c = float(b_bmi - BMI_mean)
442         eta_fix = b0 + b1*ga_c + b1q*(ga_c**2) + b2*bmi_c +
b2q*(bmi_c**2)
443         sd = eta_sd(res, ga_c, mode=guarantee_mode)
444         z = (eta_fix - float(logit(thr))) / sd
445         return float(norm.cdf(z))
446
447     probs = [prob_ge_thr_at(t, b) for t, b in zip(t_mean,
b_grid)]
448     chk = pd.DataFrame({"BMI": b_grid, "T_recommend": t_mean,
"P_ge_thr": probs})
449     chk_path = os.path.join(outdir, "guarantee_check_on_curve.
csv")
450     chk.to_csv(chk_path, index=False, encoding="utf-8-sig")

```

```

451     print(f"\n[CHECK] 推荐曲线处保障率: min={np.min(probs):.3f
452           }, median={np.median(probs):.3f} | 已导出 {chk_path}")
453
454     # 分位点推荐表
455     qs = [0.10, 0.25, 0.50, 0.75, 0.90]
456     b_q = np.quantile(b_grid, qs)
457     rows = []
458     for q, b in zip(qs, b_q):
459         t = float(np.interp(b, b_grid, t_mean))
460         lo = float(np.interp(b, b_grid, t_lo)) if np.isfinite(
461             t_lo).any() else np.nan
462         hi = float(np.interp(b, b_grid, t_hi)) if np.isfinite(
463             t_hi).any() else np.nan
464         rows.append({
465             "quantile": q, "BMI": round(float(b), 2),
466             "T_recommend_week": round(t, 2),
467             "T_lo": round(lo, 2) if np.isfinite(lo) else np.
468             nan,
469             "T_hi": round(hi, 2) if np.isfinite(hi) else np.
470             nan
471         })
472     dfq = pd.DataFrame(rows)
473     q_path = os.path.join(outdir, "recommend_by_bmi_quantiles.
474     csv")
475     dfq.to_csv(q_path, index=False, encoding="utf-8-sig")
476     print(f"[OK] 分位点推荐表已导出: {q_path}")
477
478     # 标注图
479     fig, ax = plt.subplots(figsize=(8.2, 5.2))
480     if np.isfinite(t_lo).any() and np.isfinite(t_hi).any():
481         ax.fill_between(b_grid, t_lo, t_hi, alpha=0.18,
482             linewidth=0, label="95%区间")
483     ax.plot(b_grid, t_mean, lw=2.4, label=r"$T_{\rm Bayes}(b)$")
484     for q, b in zip(qs, b_q):

```

```

478     t = np.interp(b, b_grid, t_mean)
479     ax.axvline(b, color="gray", ls=":", lw=1.0, alpha=0.6)
480     ax.plot([b], [t], "o", ms=5)
481     ax.text(b, t, f" {int(q*100)}%", va="bottom", fontsize
=10)
482     ax.set_xlabel("BMI")
483     ax.set_ylabel("推荐孕周 (周)")
484     ax.set_title("BMI → 推荐检测孕周 (贝叶斯单调曲线)")
485     ax.grid(True, alpha=0.3)
486     ax.legend()
487     fig.tight_layout()
488     fig_path = os.path.join(outdir, "fig_t_bayes_curve_marked.
png")
489     fig.savefig(fig_path, dpi=180)
490     print(f"[OK] 标注图已导出: {fig_path}")
491
492     return recommend_week
493
494
495 # ==== 误差敏感性: 把“检测误差”作为残差方差的倍率 kappa^2
====
496 def prob_ge_threshold_scaled(res, GA_mean, BMI_mean, t_week,
b_bmi, thr=0.04, kappa=1.0, mode="new"):
497     """与 prob_ge_threshold 类似, 但把残差方差按 kappa^2 放
缩; mode 控制是否计入随机效应。"""
498     b = res.fe_params
499     ga_c = t_week - GA_mean
500     bmi_c = b_bmi - BMI_mean
501     eta = (b["const"] + b["GA_c"]*ga_c + b["GA_c2"]*(ga_c
**2)
502           + b["BMI_c"]*bmi_c + b["BMI_c2"]*(bmi_c**2))
503     v = float(res.scale) * (float(kappa)**2)
504     if mode == "new":
505         vec = np.array([1.0, ga_c, float]
506         v += float(vec @ np.asarray(res.cov_re) @ vec)

```

```

507     sd = math.sqrt(max(v, 1e-12))
508     z = (eta - float(logit(thr))) / sd
509     return float(norm.cdf(z))
510
511 def compute_T_star_grid(prob_fn, b_grid, q=0.95, t_min=10.0,
512     t_max=28.0, step=0.1):
513     """在 BMI 网格上批量算 T*(b) 与是否可达标; prob_fn(t_vec,
514     b) 需支持向量化 t"""
515     T_star, reach = [], []
516     for b in b_grid:
517         grid = np.arange(t_min, t_max + 1e-9, step)
518         probs = prob_fn(grid, float(b))
519         idx = next((i for i, p in enumerate(probs) if p >= q)
520 , None)
521         if idx is None:
522             T_star.append(float(t_max)); reach.append(False)
523         else:
524             lo = max(t_min, grid[max(0, idx-2)]); hi = grid[
525 idx]
526             for _ in range(25):
527                 mid = 0.5*(lo+hi)
528                 if prob_fn(np.array([mid]), float(b))[0] >= q:
529                     hi = mid
530                 else:
531                     lo = mid
532             T_star.append(float(hi)); reach.append(True)
533     return np.array(T_star, float), np.array(reach, bool)
534
535 # -----
536 # 主流程
537 # -----
538
539 def main():
540     parser = argparse.ArgumentParser(description="第二问: 贝叶
541     斯单调曲线 + 可选分段 + 误差敏感性")

```

```

537     parser.add_argument("--data", type=str, default="附件.xlsx
538 ")
539     parser.add_argument("--sheet", type=str, default=None)
540     parser.add_argument("--idcol", type=str, default=None)
541
542     parser.add_argument("--q", type=float, default=0.95, help=
543 "保证通过率阈值")
544     parser.add_argument("--thr", type=float, default=0.04,
545 help="达标阈值（FF比例）")
546     parser.add_argument("--tmin", type=float, default=10.0)
547     parser.add_argument("--tmax", type=float, default=28.0)
548     parser.add_argument("--bmin", type=float, default=None)
549     parser.add_argument("--bmax", type=float, default=None)
550     parser.add_argument("--bstep", type=float, default=0.1)
551
552     # 贝叶斯相关
553     parser.add_argument("--method", type=str, default="vi",
554 choices=["vi", "nuts"], help="PyMC 推断方式")
555     parser.add_argument("--draws", type=int, default=1500,
556 help="后验抽样规模")
557
558     # 可选分段
559     parser.add_argument("--segment-k", type=int, default=0,
560 help=">0 则把单调曲线台阶化为 K 段（如 4 或 5）")
561     parser.add_argument("--roundto", type=float, default=0.1,
562 help="分段时段内 t 的四舍五入精度")
563     parser.add_argument("--lam-hi", type=float, default=0.05,
564 help="二分搜索λ的上界（越大段越少）")
565     parser.add_argument("--wfail", type=float, default=1.0,
566 help="失败惩罚权重 w_fail")
567     parser.add_argument("--alpha", type=float, default=0.02,
568 help="12-28周每周时间惩罚")
569     parser.add_argument("--beta", type=float, default=0.12,
570 help="≥28周每周时间惩罚")

```

```

561     # 误差敏感性
562     parser.add_argument('--err-eval', action='store_true',
563     help='是否做检测误差敏感性分析（放大/缩小残差方差）')
564     parser.add_argument('--sigma-grid', type=str, default='
0.75,1.00,1.25,1.50,2.00',
565     help='sd 倍率 kappa 列表，逗号分隔，如
0.85,1.0,1.15')
566
567     # 保证口径: typical (仅残差) / new (残差 + 随机效应)
568     parser.add_argument("--guarantee-mode", choices=["typical"
,"new"], default="typical",
569     help="typical=群体平均个体; new=新受检
者（更保守）")
570
571     args = parser.parse_args()
572
573     outdir = "out_q2_bayes"
574     ensure_dir(outdir)
575
576     # 1) 数据 & ID
577     df = read_data(args.data, sheet_name=args.sheet)
578     id_col = args.idcol or detect_id_column(df)
579     print(f"[INFO] 使用个体ID列: {id_col}")
580
581     # 2) MixedLM (与一问最优一致: logit + GA/BMI 二次, 随机截
距 + GA 随机斜率)
582     res, d2 = fit_mixedlm_best(df, id_col=id_col)
583     GA_mean, BMI_mean = float(d2["GA"].mean()), float(d2["BMI"
].mean())
584
585     # 3) 概率函数 (按口径)
586     def prob_fn_scalar(t, b):
587         return prob_ge_threshold(res, GA_mean, BMI_mean,
588         t_week=t, b_bmi=b, thr=args.
589         thr,
590         mode=args.guarantee_mode)

```

```

588
589     # 4) BMI 网格与 T*(b)
590     bmin = args.bmin if args.bmin is not None else float(d2["
BMI"].min())
591     bmax = args.bmax if args.bmax is not None else float(d2["
BMI"].max())
592     b_grid = np.round(np.arange(bmin, bmax + 1e-9, args.bstep)
, 3)
593
594     T_star = []
595     reach = []
596     for b in b_grid:
597         tstar, ok = tstar_for_bmi(prob_fn_scalar, b_bmi=float(
b), q=args.q,
598                                     t_min=args.tmin, t_max=args.
tmax, step=0.1)
599         T_star.append(tstar); reach.append(ok)
600     T_star = np.array(T_star, float)
601
602     # 5) 贝叶斯单调曲线 (无须强制分段)
603     print("[INFO] 拟合贝叶斯单调曲线 (不可用则回退 PAVA) ...")
604     T_mean, T_lo, T_hi = bayes_monotone_curve(b_grid, T_star,
method=args.method, draws=args.draws, seed=2025)
605
606     # 6) 导出与作图
607     out_csv = os.path.join(outdir, "t_bayes_curve.csv")
608     df_out = pd.DataFrame({
609         "BMI": b_grid,
610         "T_star": T_star,
611         "T_bayes_mean": T_mean,
612         "T_bayes_lo": T_lo if T_lo is not None else np.nan,
613         "T_bayes_hi": T_hi if T_hi is not None else np.nan,
614         "reachable": reach,
615     })
616     df_out.to_csv(out_csv, index=False, encoding="utf-8-sig")

```

```

617     print(f"[OK] 已导出: {out_csv}")
618
619     fig, ax = plt.subplots(figsize=(8.6,5.2))
620     ax.scatter(b_grid, T_star, s=10, alpha=0.35, label=r"$T_\backslash$
star(b)$ (最早达标) ")
621     ax.plot(b_grid, T_mean, lw=2.5, label=r"$T_{\rm Bayes}(b)$
(单调平滑, 含保守纠偏) ")
622     if T_lo is not None and T_hi is not None:
623         ax.fill_between(b_grid, T_lo, T_hi, alpha=0.18,
linewidth=0, label="95%置信带")
624     ax.set_xlabel("BMI")
625     ax.set_ylabel("孕周 (周) ")
626     ax.set_title("基于一问 MixedLM 的推荐检测时点: 贝叶斯单调
曲线")
627     ax.grid(True, alpha=0.3)
628     ax.legend()
629     fig.tight_layout()
630     out_fig = os.path.join(outdir, "fig_t_bayes_curve.png")
631     fig.savefig(out_fig, dpi=180)
632     print(f"[OK] 已导出: {out_fig}")
633
634     # 7) 回验、分位点表、标注图, 并返回查询函数
635     rec_fn = postprocess_and_export(
636         res, GA_mean, BMI_mean,
637         curve_csv=out_csv, outdir=outdir,
638         thr=args.thr, q_target=args.q,
639         guarantee_mode=args.guarantee_mode
640     )
641     print("\n[QUERY] BMI=33 的推荐孕周: ", rec_fn(33.0), "周")
642
643     # ===== 可选: 对单调推荐曲线做 K 段台阶化 (风险最优)
=====
644     if args.segment_k and args.segment_k > 0:
645         print(f"\n[SEGMENT] 目标分段数 K = {args.segment_k}")
646         # 预计算前缀概率 (候选 t 来自 T_bayes_mean 的唯一值;

```

```

kappa=1)
647     _, p_prefix = precompute_prob_prefix_from_curve(
648         res, GA_mean, BMI_mean,
649         b_grid=b_grid, T_curve=T_mean,
650         thr=args.thr, round_to=args.roundto, kappa=1.0,
651         mode=args.guarantee_mode
652     )
653     # 给定 K, 用二分搜索  $\lambda$  找到最接近 K 段的方案
654     lam_star, segs = find_lambda_for_k(
655         b_grid, T_mean, p_prefix, target_k=args.segment_k,
656         lam_lo=0.0, lam_hi=args.lam_hi, tol=1e-4,
657         round_to=args.roundto, w_fail=args.wfail,
658         alpha=args.alpha, beta=args.beta, max_iter=30
659     )
660     print(f"[SEGMENT]  $\lambda \approx \{lam\_star:.5f\}$ , 得到 {len(segs)}
段: ")
661     for idx, (i, j, t_rec) in enumerate(segs, 1):
662         print(f" 组{idx}: BMI $\in [\{b\_grid[i]:.2f\}, \{b\_grid[j]:.2f\}] \rightarrow$  推荐 {t_rec:.2f} 周 (网格数={j-i+1})")
663
664     # 段内概率函数 (用于导出表统计), 按口径
665     def prob_fn_at(t_week, b_bmi):
666         return float(_prob_vec_at_t(res, GA_mean, BMI_mean
,
667             np.array([b_bmi],
float), t_week,
668             thr=args.thr, kappa
=1.0,
669             mode=args.
guarantee_mode)[0])
670
671     export_segments_bayes(
672         outdir, b_grid, T_mean, segs, prob_fn_at,
673         alpha=args.alpha, beta=args.beta,
674         fname_csv="bmi_segments_bayes.csv",

```

```

675         fname_fig="fig_t_bayes_segments.png"
676     )
677
678     # ===== (可选) 误差敏感性: kappa 放缩残差方差 =====
679     if args.err_eval:
680         print("\n[ERR-IMPACT] 开始检测误差敏感性分析 ...")
681         kappas = [float(x) for x in str(args.sigma_grid).split
682                   (",") if str(x).strip()!=""]
683         outdir_err = os.path.join(outdir, "error_impact")
684         ensure_dir(outdir_err)
685
686         # 复用 BMI 网格
687         bmin = args.bmin if args.bmin is not None else float(
688             d2["BMI"].min())
689         bmax = args.bmax if args.bmax is not None else float(
690             d2["BMI"].max())
691         b_grid = np.round(np.arange(bmin, bmax + 1e-9, args.
692                                   bstep), 3)
693
694         # 为画 step 叠加图准备容器
695         step_curves = {} # kappa -> 台阶化后的 y(b)
696         bayes_curves = {} # kappa -> T_bayes_mean(b)
697
698         rows = []
699         for kappa in kappas:
700             print(f"[ERR-IMPACT] kappa={kappa:.2f}")
701             # 1) 概率函数 (方差×kappa^2), 支持向量化 t; 按口
702             径
703             def prob_vec_tk(t_vec, b):
704                 t_vec = np.asarray(t_vec, float)
705                 return np.array([prob_ge_threshold_scaled(res,
706                   GA_mean, BMI_mean, t, b,
707                   thr=
708                   args.thr, kappa=kappa,
709                   mode

```

```

=args.guarantee_mode)
703         for t in t_vec], float)
704
705     # 2) 计算 T*(b)
706     T_star_k, reach_k = compute_T_star_grid(
707         prob_fn=prob_vec_tk, b_grid=b_grid,
708         q=args.q, t_min=args.tmin, t_max=args.tmax,
step=0.1
709     )
710
711     # 3) 贝叶斯单调曲线 (含保守纠偏)
712     T_mean_k, T_lo_k, T_hi_k = bayes_monotone_curve(
713         b_grid, T_star_k, method=args.method, draws=
args.draws, seed=2025
714     )
715     bayes_curves[kappa] = T_mean_k.copy()
716
717     # 4) 若指定了 K 段: 在 kappa 下重新做风险最优台阶
化
718     nseg, bounds_str, trec_str, gmin, gmed = 0, "[",
"[]", np.nan, np.nan
719     if args.segment_k and args.segment_k > 0:
720         # 预计算前缀概率 (注意 kappa + 口径)
721         _, p_prefix_k =
precompute_prob_prefix_from_curve(
722             res, GA_mean, BMI_mean,
723             b_grid=b_grid, T_curve=T_mean_k,
724             thr=args.thr, round_to=args.roundto, kappa
=kappa,
725             mode=args.guarantee_mode
726         )
727     lam_star_k, segs_k = find_lambda_for_k(
728         b_grid, T_mean_k, p_prefix_k,
729         target_k=args.segment_k,
730         lam_lo=0.0, lam_hi=args.lam_hi, tol=1e-4,

```

```

731         round_to=args.roundto, w_fail=args.wfail,
732         alpha=args.alpha, beta=args.beta, max_iter
=30
733     )
734     nseg = len(segs_k)
735     # 记录边界与推荐时点
736     bounds = [(float(b_grid[i]), float(b_grid[j]))
for (i,j,_) in segs_k]
737     tlist = [float(t) for (_,_,t) in segs_k]
738     bounds_str = str(bounds)
739     trec_str = str(tlist)
740
741     # 段内保障率 (用 kappa + 口径)
742     def p_at(t_week, b_bmi):
743         return float(_prob_vec_at_t(res, GA_mean,
BMI_mean,
744                                     np.array([
b_bmi], float), t_week,
745                                     thr=args.thr,
kappa=kappa,
746                                     mode=args.
guarantee_mode)[0])
747     # 每个段把该段所有网格的概率都算一遍, 汇总 min
/median
748     all_probs = []
749     step_y = np.empty_like(b_grid, float)
750     for (i,j,t_rec) in segs_k:
751         probs_seg = [p_at(t_rec, b) for b in
b_grid[i:j+1]]
752         all_probs.extend(probs_seg)
753         step_y[i:j+1] = t_rec
754         gmin = float(np.min(all_probs)) if all_probs
else np.nan
755         gmed = float(np.median(all_probs)) if
all_probs else np.nan

```

```

756         step_curves[kappa] = step_y
757
758         # 导出该 kappa 的分段表 (可选)
759         seg_df = pd.DataFrame({
760             "BMI_min": [round(b_grid[i], 2) for (i, _, _)
761 in segs_k],
762             "BMI_max": [round(b_grid[j], 2) for (_, j, _)
763 in segs_k],
764             "t_recommend_week": [round(float(t), 2) for
765 (_, _, t) in segs_k],
766         })
767         seg_df.to_csv(os.path.join(outdir_err, f"
768 segments_kappa_{kappa:.2f}.csv"),
769                     index=False, encoding="utf-8-sig
770 ")
771
772         # 汇总一行
773         rows.append({
774             "kappa": kappa,
775             "n_segments": nseg,
776             "boundaries": bounds_str,
777             "t_rec_list": trec_str,
778             "reach_ratio_on_grid": float(np.mean(reach_k))
779         },
780         {
781             "guarantee_min_on_segments": gmin,
782             "guarantee_median_on_segments": gmed
783         })
784
785         # 5) 导出敏感性摘要
786         df_sum = pd.DataFrame(rows)
787         df_sum_path = os.path.join(outdir_err, "
788 error_impact_summary.csv")
789         df_sum.to_csv(df_sum_path, index=False, encoding="utf
790 -8-sig")
791         print(f"[ERR-IMPACT] 摘要已导出: {df_sum_path}")

```

```

783
784     # 6) 画图：曲线 & 台阶分段随 kappa 的变化
785     # 6.1 曲线叠加图
786     fig, ax = plt.subplots(figsize=(8.6,5.0))
787     for kappa, y in bayes_curves.items():
788         ax.plot(b_grid, y, lw=1.8, alpha=0.9, label=f"
kappa={kappa:g}")
789         ax.set_xlabel("BMI"); ax.set_ylabel("推荐孕周（周）")
790         ax.set_title("误差放缩 kappa 下的单调推荐曲线对比")
791         ax.grid(True, alpha=0.3); ax.legend(ncol=2, fontsize
=9)
792         fig.tight_layout()
793         fig.savefig(os.path.join(outdir_err, "
fig_curves_vs_kappa.png"), dpi=170)
794
795     # 6.2 台阶分段叠加图（仅在设了 K 才有）
796     if step_curves:
797         fig, ax = plt.subplots(figsize=(8.6,5.0))
798         for kappa, y in step_curves.items():
799             ax.step(b_grid, y, where="mid", lw=2.2, label=
f"kappa={kappa:g}")
800             ax.set_xlabel("BMI"); ax.set_ylabel("推荐孕周
（周）")
801             ax.set_title(f"K={args.segment_k} 段的台阶化结果随
kappa 的变化")
802             ax.grid(True, alpha=0.3); ax.legend(ncol=2,
fontsize=9)
803             fig.tight_layout()
804             fig.savefig(os.path.join(outdir_err, "
fig_segments_vs_kappa.png"), dpi=170)
805
806         print("[ERR-IMPACT] 完成。")
807
808 if __name__ == "__main__":
809     main()

```

q3.py

```
1 # -*- coding: utf-8 -*-
2 """
3 q3.py    第三问（接入 PyMC 的贝叶斯单调回归 + 批量 kappa + 绘
          图 + 方案A(G 作用研究)）
4 -----
5 - 单调化支持: Bayes(PyMC) / PAVA; 若无 PyMC 自动回退 PAVA
6 - 支持一次输入多个 kappa: --kappa-batch "1.00,1.15,1.25"
7   · 批量模式默认每个 kappa 输出到 outdir/kappa_x.xx 子目录 (--
     no-batch-subdir 可关闭)
8   · 只在第一轮画“灵敏度图”，避免重复计算
9 - 支持画 95% 置信带 (--show-cred, 仅 Bayes 时有效)
10 - 方案A: 新增 --omit-g 与 --g-grid "0,1,2,3":
11   · omit-g: 固定效应中不加入 G (怀孕次数)，用于与“含G”做对
     照
12   · g-grid: 在模型含G时，指定一组 G 水平，分别计算与绘制 t*(
     BMI) 单调曲线，并绘制相邻 G 的  $\Delta t^*$  曲线
13 依赖: pandas, numpy, statsmodels, scikit-learn, scipy,
     matplotlib
14 可选: pymc>=4 (启用贝叶斯单调回归)
15 """
16
17 import argparse, os, re, warnings, math, pathlib
18 from dataclasses import dataclass
19 from typing import List, Tuple, Optional
20
21 import numpy as np
22 import pandas as pd
23 from scipy.stats import norm
24 from sklearn.isotonic import IsotonicRegression
25
26 # ----- statsmodels -----
27 try:
28     import statsmodels.api as sm
```

```

29     from statsmodels.regression.linear_model import OLS
30 except Exception as e:
31     raise RuntimeError("需要安装 statsmodels: pip install
    statsmodels") from e
32
33 # ----- optional: PyMC -----
34 HAS_PYMC = False
35 try:
36     import pymc as pm
37     try:
38         import pytensor.tensor as pt
39     except Exception:
40         import aesara.tensor as pt
41     HAS_PYMC = True
42 except Exception:
43     HAS_PYMC = False
44
45
46 # ===== 基础清洗 =====
47 def logit(p: np.ndarray, eps: float = 1e-6) -> np.ndarray:
48     p = np.clip(p, eps, 1 - eps)
49     return np.log(p / (1 - p))
50
51 def try_get_col(df: pd.DataFrame, candidates: list) ->
    Optional[str]:
52     for c in candidates:
53         if c in df.columns:
54             return c
55     return None
56
57 def ff_to_fraction(val):
58     if val is None or (isinstance(val, float) and np.isnan(val
    )):
59         return np.nan
60     if isinstance(val, (int, float, np.number)):

```

```

61     x = float(val)
62     if 0 < x < 1:
63         return x
64     if 1.5 <= x <= 100:
65         return x/100.0
66     return np.nan
67 s = str(val).strip().lower()
68 if s.endswith('%'):
69     try:
70         return float(s[:-1])/100.0
71     except:
72         return np.nan
73 m = re.match(r'^\d+(?:\.\d+)?$', s)
74 if m:
75     x = float(s)
76     if 0 < x < 1:
77         return x
78     if 1.5 <= x <= 100:
79         return x/100.0
80     return np.nan
81
82
83
84 def phi_time(t_week: float, alpha: float = 0.02, beta: float =
85             0.12, t_ref: float = 28.0) -> float:
86     """相对 28 周的时间惩罚：提前用 alpha，滞后用 beta（默认滞
87     后更贵）。"""
88     return (t_ref - t_week) * alpha if t_week < t_ref else (
89         t_week - t_ref) * beta
90
91 def compute_segment_metrics(b_grid, segs, prob_fn_at, alpha:
92                             float = 0.02, beta: float = 0.12):
93     """
94     返回每段：BMI_min/max、推荐周、p_ge_thr_mean、risk_mean、
95     n_grid。

```

```

91     risk_mean = mean( (1-p) + phi_time(t_rec) ), 与第二问口径
    一致。
92     """
93     import numpy as np
94     rows = []
95     for s in segs:
96         i, j, t_rec = (s.start_idx, s.end_idx, float(s.y_hat))
97         bmin, bmax = float(b_grid[i]), float(b_grid[j])
98         b_list = b_grid[i:j+1]
99         probs = [float(prob_fn_at(t_rec, float(b))) for b in
    b_list]
100         p_mean = float(np.mean(probs))
101         r_mean = float(np.mean([(1.0 - p) + phi_time(t_rec,
    alpha, beta) for p in probs]))
102         rows.append(dict(
103             BMI_min=round(bmin, 2), BMI_max=round(bmax, 2),
104             t_recommend_week=round(t_rec, 2),
105             p_ge_thr_mean=round(p_mean, 4),
106             risk_mean=round(r_mean, 4),
107             n_grid=int(len(b_list)),
108         ))
109     return rows
110
111
112 def ga_to_weeks(val):
113     if val is None or (isinstance(val, float) and np.isnan(val
    ))):
114         return np.nan
115     if isinstance(val, (int, float, np.number)):
116         return float(val)
117     s = str(val).strip().lower()
118     s = (s.replace('周', 'w').replace('天', 'd').replace('日', 'w
    ').replace('日', 'd').replace('+', '+'))
119     m = re.match(r'^(\d+)\s*w?\s*(?:\+|\s*)\s*(\d{1,2})\s*d?$'
    , s) # 11w+6 / 11+6

```

```

120     if m:
121         return int(m.group(1)) + int(m.group(2))/7.0
122     m = re.match(r'^(\d+)\s*w$', s)
123         # 11w
124     if m:
125         return float(m.group(1))
126     m = re.match(r'^(\d+)w(\d{1,2})d$', s)
127         # 11w6d
128     if m:
129         return int(m.group(1)) + int(m.group(2))/7.0
130     m = re.match(r'^\d+(?:\.\d+)?$', s)
131         # 14 / 14.5
132     if m:
133         return float(s)
134     return np.nan
135
136 def to_numeric_loose(series: pd.Series) -> pd.Series:
137     s_num = pd.to_numeric(series, errors='coerce')
138     need = s_num.isna()
139     if need.any():
140         s_str = series.astype(str).str.extract(r'(\d+)',
141         expand=False)
142         s_fill = pd.to_numeric(s_str, errors='coerce')
143         s_num = s_num.where(~need, s_fill)
144     return s_num
145
146 # ===== MixedLM 拟合 =====
147 def fit_baseline_model(df: pd.DataFrame,
148     col_ff: str, col_ga: str, col_bmi: str,
149     col_id: Optional[str],
150     add_age_gp: bool,
151     col_age: Optional[str], col_g: Optional
152     [str], col_p: Optional[str],
153     omit_g: bool = False):

```

```

150     y = logit(df[col_ff].to_numpy())
151     GA = df[col_ga].to_numpy().astype(float)
152     BMI = df[col_bmi].to_numpy().astype(float)
153     GA_c = GA - np.nanmean(GA)
154     BMI_c = BMI - np.nanmean(BMI)
155
156     X_list = [GA_c, GA_c**2, BMI_c, BMI_c**2]
157     age_gp_cols = []
158     if add_age_gp:
159         if col_age and col_age in df.columns:
160             age_gp_cols.append('AGE')
161             a = pd.to_numeric(df[col_age], errors='coerce').
to_numpy(dtype=float)
162             X_list.append(a - np.nanmean(a))
163             if (not omit_g) and col_g and col_g in df.columns:
164                 age_gp_cols.append('G')
165                 g = pd.to_numeric(df[col_g], errors='coerce').
to_numpy(dtype=float)
166                 X_list.append(g - np.nanmean(g))
167                 if col_p and col_p in df.columns:
168                     age_gp_cols.append('P')
169                     p = pd.to_numeric(df[col_p], errors='coerce').
to_numpy(dtype=float)
170                     X_list.append(p - np.nanmean(p))
171
172     X = np.column_stack(X_list)
173     X = sm.tools.tools.add_constant(X, has_constant='add')
174
175     design = {
176         'GA_mean': float(np.nanmean(GA)),
177         'BMI_mean': float(np.nanmean(BMI)),
178         'age_means': {
179             'AGE': float(pd.to_numeric(df[col_age], errors='
coerce').mean()) if (col_age and col_age in df.columns) else
0.0,

```

```

180         'G': float(pd.to_numeric(df[col_g], errors='coerce
').mean()) if (col_g and col_g in df.columns) else 0.0,
181         'P': float(pd.to_numeric(df[col_p], errors='coerce
').mean()) if (col_p and col_p in df.columns) else 0.0,
182     },
183     'age_gp_cols': age_gp_cols
184 }
185
186 if col_id and col_id in df.columns:
187     groups = df[col_id].astype('category')
188     Z = np.column_stack([np.ones_like(GA_c), GA_c])
189     try:
190         with warnings.catch_warnings():
191             warnings.simplefilter("ignore")
192             md = sm.MixedLM(endog=y, exog=X, groups=groups
, exog_re=Z)
193             mfit = md.fit(method='lbfgs', reml=False,
maxiter=500, disp=False)
194             return 'MixedLM', mfit, design
195     except Exception as e:
196         warnings.warn(f"MixedLM 拟合失败，回退 OLS(HC3): {
e}")
197     return 'OLS', OLS(y, X).fit(cov_type='HC3'), design
198
199
200 # ===== 概率与 t* =====
201 def mu_at_t_bmi(result, model_type, design, t_week: float, bmi
: float,
202                 age_val: Optional[float], g_val: Optional[
float], p_val: Optional[float]) -> float:
203     GA_c = float(t_week - design['GA_mean'])
204     BMI_c = float(bmi - design['BMI_mean'])
205     X = [1.0, GA_c, GA_c**2, BMI_c, BMI_c**2]
206     if 'age_gp_cols' in design and design['age_gp_cols']:
207         means = design['age_means']

```

```

208         if 'AGE' in design['age_gp_cols'] and age_val is not
None:
209             X.append(age_val - means.get('AGE', 0.0))
210         if 'G' in design['age_gp_cols'] and g_val is not None:
211             X.append(g_val - means.get('G', 0.0))
212         if 'P' in design['age_gp_cols'] and p_val is not None:
213             X.append(p_val - means.get('P', 0.0))
214         beta = result.fe_params if model_type == 'MixedLM' else
result.params
215         return float(np.dot(beta, np.asarray(X, float)))
216
217 def sd_at_t(result, model_type, t_week: float, design,
var_mode: str='new', kappa: float=1.0) -> float:
218     v = float(result.scale) * float(kappa)**2
219     if (var_mode == 'new') and (model_type == 'MixedLM') and
hasattr(result, 'cov_re'):
220         GA_c = float(t_week - design['GA_mean'])
221         vec = np.asarray([1.0, GA_c], float)
222         try:
223             cov_re = np.asarray(result.cov_re, float)
224             v += float(vec @ cov_re @ vec)
225         except Exception:
226             pass
227         return math.sqrt(max(v, 1e-12))
228
229 def prob_ge_thr(result, model_type, design, t_week: float, bmi
: float, thr: float,
age_val: Optional[float], g_val: Optional[
230 float], p_val: Optional[float],
var_mode: str='new', kappa: float=1.0) ->
231 float:
232     eta = mu_at_t_bmi(result, model_type, design, t_week, bmi,
age_val, g_val, p_val)
233     sd = sd_at_t(result, model_type, t_week, design, var_mode=
var_mode, kappa=kappa)

```

```

234     return float(norm.cdf((eta - float(logit(np.array([thr]))
235                               [0])) / sd))
236
237 def tstar_for_bmi(result, model_type, design, bmi, q, tmin,
238                  tmax, step, thr,
239                  age_val, g_val, p_val, var_mode='new', kappa
240                  =1.0) -> float:
241     grid = np.arange(tmin, tmax + 1e-9, step)
242     probs = [prob_ge_thr(result, model_type, design, t, bmi,
243                          thr, age_val, g_val, p_val, var_mode, kappa) for t in grid]
244     idx = next((i for i, p in enumerate(probs) if p >= q),
245               None)
246     if idx is None:
247         return float(tmax)
248     lo = max(tmin, grid[max(0, idx-2)])
249     hi = grid[idx]
250     for _ in range(25):
251         mid = 0.5*(lo+hi)
252         if prob_ge_thr(result, model_type, design, mid, bmi,
253                      thr, age_val, g_val, p_val, var_mode, kappa) >= q:
254             hi = mid
255         else:
256             lo = mid
257     return float(hi)
258
259 # ===== 台阶化 =====
260 @dataclass
261 class Segment:
262     start_idx: int
263     end_idx: int
264     y_hat: float
265
266 def stepwise_segments_L2(x: np.ndarray, y: np.ndarray, K: int,
267                        min_width: Optional[int]=None) -> List[Segment]:

```

```

262     order = np.argsort(x, kind="mergesort")
263     x, y = x[order], y[order]
264     n = len(y)
265     K = int(min(max(1, K), n))
266     if min_width is None:
267         min_width = max(1, n // (K * 4))
268     Wy = np.cumsum(y); W = np.cumsum(np.ones_like(y)); yy = np
    .cumsum(y**2)
269     def cost(i,j):
270         sw = W[j] - (W[i-1] if i>0 else 0.0)
271         swy= Wy[j]- (Wy[i-1]if i>0 else 0.0)
272         syy= yy[j]- (yy[i-1]if i>0 else 0.0)
273         return float(syy - (swy**2)/max(sw,1e-12))
274     def mean(i,j):
275         sw = W[j] - (W[i-1] if i>0 else 0.0)
276         swy= Wy[j]- (Wy[i-1]if i>0 else 0.0)
277         return float(swy/max(sw,1e-12))
278     C = np.full((n,n), np.inf); M = np.zeros((n,n))
279     for i in range(n):
280         j0 = max(i, i+min_width-1)
281         for j in range(j0, n):
282             C[i,j] = cost(i,j); M[i,j] = mean(i,j)
283     dp = np.full((K+1,n), np.inf); ptr = -np.ones((K+1,n), int
    )
284     for j in range(min_width-1, n):
285         dp[1,j] = C[0,j]; ptr[1,j] = -1
286     for k in range(2, K+1):
287         for j in range(k*min_width-1, n):
288             imin = (k-1)*min_width-1; best = np.inf; arg=-1
289             for i in range(imin, j-min_width+1):
290                 cand = dp[k-1,i] + C[i+1,j]
291                 if cand < best:
292                     best, arg = cand, i
293             dp[k,j]=best; ptr[k,j]=arg
294     segs=[]; k=K; j=n-1

```

```

295     while k>=1:
296         i=ptr[k,j]; st=0 if i<0 else i+1; ed=j
297         segs.append(Segment(st,ed,float(M[st,ed]))); j=i; k-=1
298     segs.reverse()
299     return segs
300
301 def enforce_monotone_on_segments(segs: List[Segment]) -> List[
    Segment]:
302     if not segs:
303         return segs
304     n_list = [seg.end_idx-seg.start_idx+1 for seg in segs]
305     y_list = [seg.y_hat for seg in segs]
306     idx = list(range(len(segs)))
307     iso = IsotonicRegression(increasing=True, out_of_bounds='
clip')
308     y_mono = iso.fit_transform(idx, y_list, sample_weight=
n_list)
309     for seg, ym in zip(segs, y_mono):
310         seg.y_hat = float(ym)
311     return segs
312
313 def summarize_segments(x_sorted: np.ndarray, y_sorted: np.
ndarray, segs: List[Segment], roundto=0.1) -> pd.DataFrame:
314     rows=[]
315     for s in segs:
316         bmin, bmax = float(x_sorted[s.start_idx]), float(
x_sorted[s.end_idx])
317         n = int(s.end_idx - s.start_idx + 1)
318         t = float(s.y_hat)
319         if roundto and roundto>0:
320             t = round(t/roundto)*roundto
321         rows.append({"BMI_min": bmin, "BMI_max": bmax, "n": n,
    "t_star_rec": t})
322     return pd.DataFrame(rows)
323

```

```

324 def assign_segments(x_sorted: np.ndarray, segs: List[Segment])
    -> np.ndarray:
325     seg_id = np.empty(len(x_sorted), int); cur=1
326     for s in segs:
327         seg_id[s.start_idx:s.end_idx+1] = cur; cur+=1
328     return seg_id
329
330
331 # ===== 单调化: Bayes / PAVA =====
332 def bayes_monotone_curve(b_sorted: np.ndarray,
333                          y_obs: np.ndarray,
334                          method: str = 'vi',
335                          vi_steps: int = 12000,
336                          draws: int = 1000,
337                          tune: int = 1000,
338                          chains: int = 2,
339                          target_accept: float = 0.9,
340                          seed: int = 42) -> Tuple[np.ndarray,
np.ndarray, np.ndarray]:
341     """
342     使用贝叶斯单调回归（累积非负增量）对 (b_sorted, y_obs) 拟
    合单调曲线：
343          $T(b_k) = \mu_0 + \text{cumsum}(\text{softplus}(z_k) * s * \Delta b_k)$ 
344          $y_k \sim \text{Normal}(T(b_k), \text{sigma})$ 
345     返回：
346         T_mean, T_lo, T_hi          分别为后验均值与 95% 置信带的
    下/上界（长度 = len(b_sorted)）
347     """
348     try:
349         import pymc as pm
350     try:
351         import pytensor.tensor as pt
352     except Exception:
353         import aesara.tensor as pt
354     except Exception as e:

```

```

355         raise RuntimeError("未检测到 PyMC, 请改用 --mono-
method pava 或安装 pymc>=4") from e
356
357     n = len(b_sorted)
358     if n != len(y_obs):
359         raise ValueError(f"b_sorted 与 y_obs 长度不一致: {n}
vs {len(y_obs)}")
360
361     db = np.diff(b_sorted)
362     if db.size == 0:
363         db = np.array([1.0])
364     step = np.concatenate([[np.median(db)], db])
365
366     try:
367         sp = pt.nnet.softplus
368     except Exception:
369         def sp(x): return pt.log1p(pt.exp(x))
370
371     with pm.Model() as m:
372         mu0 = pm.Normal('mu0', mu=np.nanmedian(y_obs), sigma
=2.0)
373         s = pm.HalfNormal('s', sigma=0.5)
374         z = pm.Normal('z', 0.0, 1.0, shape=n)
375         inc = pm.Deterministic('inc', sp(z) * s * step)      #
非负增量 * 步长
376         T = pm.Deterministic('T', mu0 + pm.math.cumsum(inc)) #
单调曲线
377         sigma = pm.HalfNormal('sigma', sigma=0.4)
378         pm.Normal('y', mu=T, sigma=sigma, observed=y_obs)
379
380         if method == 'vi':
381             approx = pm.fit(n=vi_steps, method='advi',
random_seed=seed, progressbar=False)
382             samples = approx.sample(draws=draws, random_seed=
seed)

```

```

383     # 取出 T 的样本
384     T_samps = None
385     try:
386         da = samples.posterior['T']
387         dims = list(da.dims)
388         if 'chain' in dims and 'draw' in dims:
389             pt_dim = [d for d in dims if d not in ('
chain','draw'')]
390             if len(pt_dim) == 1:
391                 da = da.transpose('chain','draw',
pt_dim[0])
392                 T_samps = da.values
393             except Exception:
394                 T_samps = samples.get_values('T', combine=True
)
395         else:
396             idata = pm.sample(draws=draws, tune=tune, chains=
chains,
397                               target_accept=target_accept,
random_seed=seed, cores=1)
398             da = idata.posterior['T']
399             dims = list(da.dims)
400             if 'chain' in dims and 'draw' in dims:
401                 pt_dim = [d for d in dims if d not in ('chain'
,'draw'')]
402                 if len(pt_dim) == 1:
403                     da = da.transpose('chain','draw', pt_dim
[0])
404                     T_samps = da.values
405
406     T_samps = np.asarray(T_samps)
407     n_points = n
408
409     def to_draws_points(arr: np.ndarray, n_points: int) -> np.
ndarray:

```

```

410     if arr.ndim == 1:
411         if arr.shape[0] == n_points:
412             return arr[None, :]
413         raise ValueError(f"采样形状异常: {arr.shape}")
414     if arr.ndim == 2:
415         if arr.shape[-1] == n_points:
416             return arr
417         if arr.shape[0] == n_points:
418             return arr.T
419         if arr.shape[0] > arr.shape[1] and arr.shape[1] !=
n_points:
420             arr = arr.T
421             if arr.shape[-1] != n_points:
422                 raise ValueError(f"采样形状异常: {arr.shape}
vs n_points={n_points}")
423             return arr
424     if arr.ndim >= 3:
425         axes = list(range(arr.ndim))
426         cand = [i for i, s in enumerate(arr.shape) if s ==
n_points]
427         if not cand:
428             if arr.shape[-1] != n_points:
429                 raise ValueError(f"采样形状异常: {arr.
shape} vs n_points={n_points}")
430             point_axis = arr.ndim - 1
431         else:
432             point_axis = cand[-1]
433         if point_axis != arr.ndim - 1:
434             axes.remove(point_axis)
435             axes.append(point_axis)
436         arr = np.transpose(arr, axes)
437         return arr.reshape(-1, n_points)
438         raise ValueError(f"采样维度异常: ndim={arr.ndim}")
439
440 T_samps = to_draws_points(T_samps, n_points)

```

```

441     T_mean = T_samps.mean(axis=0)
442     T_lo = np.quantile(T_samps, 0.025, axis=0)
443     T_hi = np.quantile(T_samps, 0.975, axis=0)
444
445     if not (len(T_mean) == len(T_lo) == len(T_hi) == len(
b_sorted)):
446         raise RuntimeError(f"返回长度不一致: mean={len(T_mean)
}, lo={len(T_lo)}, hi={len(T_hi)}, b={len(b_sorted)}")
447     return T_mean, T_lo, T_hi
448
449
450 def pava_monotone_curve(b_sorted: np.ndarray, y_obs: np.
ndarray) -> np.ndarray:
451     iso = IsotonicRegression(increasing=True, out_of_bounds='
clip')
452     return iso.fit_transform(b_sorted, y_obs)
453
454
455 # ===== 绘图 (通用/灵敏度) =====
456 def _mpl_setup():
457     import matplotlib.pyplot as plt
458     try:
459         plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft
YaHei', 'Arial', 'DejaVu Sans']
460         plt.rcParams['axes.unicode_minus'] = False
461     except Exception:
462         pass
463     return plt
464
465 def plot_curve(figpath_png: str, b_grid: np.ndarray, T_star:
np.ndarray,
466                T_curve: np.ndarray, dpi: int=180, save_pdf:
bool=False,
467                cred_lo: Optional[np.ndarray]=None, cred_hi:
Optional[np.ndarray]=None):

```

```

468     plt = _mpl_setup()
469     fig = plt.figure(figsize=(7.2, 4.6))
470     plt.scatter(b_grid, T_star, s=10, alpha=0.55, label='T*(
BMI) raw')
471     if cred_lo is not None and cred_hi is not None:
472         plt.fill_between(b_grid, cred_lo, cred_hi, alpha=0.18,
label='95% CI')
473     plt.plot(b_grid, T_curve, linewidth=2.2, label='Monotone
curve')
474     plt.xlabel('BMI'); plt.ylabel('Gestational week')
475     plt.title('T*(BMI) & Monotone Curve')
476     plt.legend(loc='best', frameon=False)
477     fig.tight_layout()
478     fig.savefig(figpath_png, dpi=dpi)
479     if save_pdf:
480         fig.savefig(figpath_png.replace('.png', '.pdf'))
481     plt.close(fig)
482
483 def plot_segments(figpath_png: str, b_sorted: np.ndarray,
T_sorted: np.ndarray,
484                 segs: List[Segment], dpi: int=180, save_pdf:
bool=False):
485     plt = _mpl_setup()
486     fig = plt.figure(figsize=(7.2, 4.6))
487     plt.plot(b_sorted, T_sorted, linewidth=1.9, alpha=0.95,
label='Monotone curve')
488     for s in segs:
489         x = b_sorted[s.start_idx:s.end_idx+1]
490         y = np.full_like(x, segs[segs.index(s)].y_hat, dtype=
float)
491         plt.plot(x, y, linewidth=3.0)
492         plt.xlabel('BMI'); plt.ylabel('Gestational week')
493         plt.title('Monotone Curve with K-step Approximation')
494         plt.legend(['Monotone curve', 'Steps'], loc='best', frameon
=False)

```

```

495     fig.tight_layout()
496     fig.savefig(figpath_png, dpi=dpi)
497     if save_pdf:
498         fig.savefig(figpath_png.replace('.png', '.pdf'))
499     plt.close(fig)
500
501 def plot_sensitivity(figpath_png: str, kappas: List[float],
502                     compute_T_curve_fn,
503                     dpi: int=180, save_pdf: bool=False):
504     """compute_T_curve_fn(kappa) -> (b_grid, T_curve)"""
505     plt = _mpl_setup()
506     fig = plt.figure(figsize=(7.2, 4.6))
507     for k in kappas:
508         b, T = compute_T_curve_fn(k)
509         plt.plot(b, T, linewidth=2.0, label=f'kappa={k:g}')
510     plt.xlabel('BMI'); plt.ylabel('Gestational week')
511     plt.title('Sensitivity: Monotone Curves vs kappa')
512     plt.legend(loc='best', frameon=False)
513     fig.tight_layout()
514     fig.savefig(figpath_png, dpi=dpi)
515     if save_pdf:
516         fig.savefig(figpath_png.replace('.png', '.pdf'))
517     plt.close(fig)
518
519 # 方案A: 按G分层的两张图
520 def plot_curves_vs_g(figpath_png: str, b_grid: np.ndarray,
521                     curves_by_g, dpi: int=180, save_pdf: bool=False):
522     plt = _mpl_setup()
523     fig = plt.figure(figsize=(9.6, 5.2))
524     for g_val, curve in sorted(curves_by_g, key=lambda x: x[0]):
525         plt.plot(b_grid, curve, linewidth=2.2, label=f'G={g_val:g}')
526     plt.xlabel('BMI'); plt.ylabel('推荐孕周 (周)')
527     plt.title('不同 G 水平的 t*(BMI) 单调推荐曲线对比')

```

```

526 plt.legend(frameon=False); plt.grid(alpha=0.25)
527 fig.tight_layout(); fig.savefig(figpath_png, dpi=dpi)
528 if save_pdf: fig.savefig(figpath_png.replace('.png', '.pdf'
529 ))
529 plt.close(fig)
530
531 def plot_delta_tstar_vs_g(figpath_png: str, b_grid: np.ndarray
532 , curves_by_g, dpi: int=180, save_pdf: bool=False):
533     """对相邻G作差:  $\Delta t_* = t_*(G_{i+1}) - t_*(G_i)$ """
534     plt = _mpl_setup()
535     fig = plt.figure(figsize=(9.6, 5.2))
536     curves_by_g = sorted(curves_by_g, key=lambda x: x[0])
537     for (g1, c1), (g2, c2) in zip(curves_by_g[:-1],
538 curves_by_g[1:]):
539         delta = np.asarray(c2) - np.asarray(c1)
540         plt.plot(b_grid, delta, linewidth=2.0, label=f' $\Delta t_*$ : G
541 ={g1:g}→{g2:g}')
542     plt.xlabel('BMI'); plt.ylabel(' $\Delta t_*$  (周)')
543     plt.title('相邻 G 水平的推荐周数差值 ( $\Delta t_*$ )')
544     plt.legend(frameon=False); plt.grid(alpha=0.25)
545     fig.tight_layout(); fig.savefig(figpath_png, dpi=dpi)
546     if save_pdf: fig.savefig(figpath_png.replace('.png', '.pdf'
547 ))
548     plt.close(fig)
549
550 # ===== 主流程 =====
551 def main():
552     ap = argparse.ArgumentParser()
553     ap.add_argument('--xlsx', required=True)
554     ap.add_argument('--sheet', default=None)
555     ap.add_argument('--id-col', dest='id_col', default=None)
556
557     ap.add_argument('--col-ff', dest='col_ff', default=None)
558     ap.add_argument('--col-ga', dest='col_ga', default=None)

```

```

556 ap.add_argument('--col-bmi', dest='col_bmi', default=None)
557 ap.add_argument('--col-age', dest='col_age', default=None)
558 ap.add_argument('--col-g', dest='col_g', default=None)
559 ap.add_argument('--col-p', dest='col_p', default=None)
560
561 ap.add_argument('--thr', type=float, default=0.04)
562 ap.add_argument('--q', type=float, default=0.95)
563 ap.add_argument('--tmin', type=float, default=12.0)
564 ap.add_argument('--tmax', type=float, default=28.0)
565 ap.add_argument('--tstep', type=float, default=0.25)
566 ap.add_argument('--bmi-step', dest='bmi_step', type=float,
567                 default=0.1)
568
569 ap.add_argument('--var-mode', dest='var_mode', choices=[
570                 'new', 'typical'], default='new',
571                 help='new: scale*kappa^2 + RE方差; typical
572                 : 仅scale*kappa^2')
573 ap.add_argument('--kappa', type=float, default=1.0, help='
574                 误差放大倍数')
575
576 ap.add_argument('--mono-curve', dest='mono_curve', action=
577                 'store_true', default=True)
578 ap.add_argument('--no-mono-curve', dest='mono_curve',
579                 action='store_false')
580 ap.add_argument('--mono-steps', dest='mono_steps', action=
581                 'store_true', default=True)
582 ap.add_argument('--no-mono-steps', dest='mono_steps',
583                 action='store_false')
584
585 ap.add_argument('--bmi-min', dest='bmi_min', type=float,
586                 default=None)
587 ap.add_argument('--bmi-max', dest='bmi_max', type=float,
588                 default=None)
589
590 ap.add_argument('--K', type=int, default=4)

```

```

581     ap.add_argument('--roundto', type=float, default=0.1)
582     ap.add_argument('--min-width', dest='min_width', type=int,
583                     default=None)
584
585     # 绘图
586     ap.add_argument('--plots', action='store_true', help='生成
587     三张图')
588     ap.add_argument('--figdir', default=None, help='图片输出目
589     录（默认与 outdir 相同）')
590     ap.add_argument('--fig-dpi', dest='fig_dpi', type=int,
591                     default=180)
592     ap.add_argument('--save-pdf', dest='save_pdf', action='
593     store_true', help='同时保存 PDF')
594     ap.add_argument('--kappa-grid', dest='kappa_grid', default
595     ='0.9,1.0,1.15,1.25',
596                     help='灵敏度图的 kappa 列表，逗号分隔')
597     ap.add_argument('--show-cred', action='store_true', help='
598     bayes 单调化时显示 95% 置信带')
599
600     # 单调化控制
601     default_mono = 'bayes' if HAS_PYMC else 'pava'
602     ap.add_argument('--mono-method', choices=['bayes', 'pava'],
603                     default=default_mono,
604                     help='单调化方式（默认：若装了 PyMC 则
605     bayes，否则 pava）')
606     ap.add_argument('--envelope', action='store_true', default
607     =True,
608                     help='是否取上包络 max(T_mono, T_star)')
609     ap.add_argument('--no-envelope', dest='envelope', action='
610     store_false')
611
612     # Bayes 细节
613     ap.add_argument('--bayes-method', choices=['vi', 'nuts'],
614                     default='vi',

```

```

604         help='贝叶斯推断方式: vi(ADVI) 或 nuts')
605     ap.add_argument('--vi-steps', type=int, default=12000)
606     ap.add_argument('--draws', type=int, default=1000)
607     ap.add_argument('--tune', type=int, default=1000)
608     ap.add_argument('--chains', type=int, default=2)
609     ap.add_argument('--target-accept', type=float, default
610 =0.90)
611     ap.add_argument('--seed', type=int, default=42)
612
613     # 批量 kappa
614     ap.add_argument('--kappa-batch', dest='kappa_batch',
615 default=None,
616         help='一次性跑多个 kappa, 逗号分隔, 例如
617 "1.00,1.15,1.25"')
618     ap.add_argument('--batch-subdir', action='store_true',
619         help='批量模式: 每个 kappa 输出到 outdir/
620 kappa_x.xx 子目录 (默认开启)')
621     ap.add_argument('--no-batch-subdir', dest='batch_subdir',
622 action='store_false')
623     ap.set_defaults(batch_subdir=True)
624
625     # ==== 方案A: G 效应研究 ====
626     ap.add_argument('--omit-g', action='store_true',
627         help='不把 G 纳入固定效应 (用于和含G模型作
628 对照)')
629     ap.add_argument('--g-grid', default=None,
630         help='按 G 分层绘制/导出: 例如 "0,1,2,3";
631 仅在模型含G时有意义')
632     ap.add_argument('--alpha', type=float, default=0.02, help=
633 '提前惩罚/周 (风险指数)')
634     ap.add_argument('--beta', type=float, default=0.12, help=
635 '滞后惩罚/周 (风险指数)')
636
637     args = ap.parse_args()
638     os.makedirs(args.outdir, exist_ok=True)

```

```

630     figdir = args.figdir or args.outdir
631     pathlib.Path(figdir).mkdir(parents=True, exist_ok=True)
632
633     # 读表
634     xls = pd.ExcelFile(args.xlsx)
635     sheet_name = args.sheet or (next((s for s in xls.
sheet_names if ('男胎' in s or '女胎' in s)), xls.
sheet_names[0]))
636     df = pd.read_excel(args.xlsx, sheet_name=sheet_name)
637     df.columns = [str(c).strip() for c in df.columns]
638
639     # 列名
640     col_id = args.id_col or try_get_col(df, ['孕妇代码', 'ID'
, '受试者ID', '样本ID'])
641     col_ff = args.col_ff or try_get_col(df, ['Y染色体浓度', '
FF', 'fetal_fraction', 'FF%', 'FF(%)', 'Y染色体FF'])
642     col_ga = args.col_ga or try_get_col(df, ['检测孕周', '孕
周', '孕周数', '孕周(周)', 'GA'])
643     col_bmi = args.col_bmi or try_get_col(df, ['BMI', '体质指数
', '孕妇BMI'])
644     col_age = args.col_age or try_get_col(df, ['AGE', '年龄'])
645     col_g = args.col_g or try_get_col(df, ['G', '怀孕次数',
'孕次'])
646     col_p = args.col_p or try_get_col(df, ['P', '生产次数',
'产次'])
647
648     if any(c is None for c in [col_ff, col_ga, col_bmi]):
649         raise ValueError(f"核心列无法识别, 请手动指定: FF({
col_ff}), GA({col_ga}), BMI({col_bmi})")
650
651     # 清洗
652     core_cols = [c for c in [col_ff, col_ga, col_bmi, col_age,
col_g, col_p, col_id] if c]
653     d0 = df[core_cols].copy().replace([np.inf, -np.inf], np.
nan)

```

```

654     d0[col_ff] = d0[col_ff].apply(ff_to_fraction)
655     d0[col_ga] = d0[col_ga].apply(ga_to_weeks)
656     if col_age and col_age in d0.columns: d0[col_age] =
to_numeric_loose(d0[col_age])
657     if col_g and col_g in d0.columns: d0[col_g] =
to_numeric_loose(d0[col_g])
658     if col_p and col_p in d0.columns: d0[col_p] =
to_numeric_loose(d0[col_p])
659     d0 = d0.dropna(subset=[col_ff, col_ga, col_bmi])
660     d0 = d0[(d0[col_ff] > 0) & (d0[col_ff] < 1)]
661
662     # 拟合
663     add_age_gp = any([c in d0.columns for c in [col_age, col_g
, col_p]])
664     model_type, result, design = fit_baseline_model(
665         d0, col_ff, col_ga, col_bmi, col_id,
666         add_age_gp, col_age, col_g, col_p,
667         omit_g=args.omit_g
668     )
669
670     # BMI 栅格
671     if args.bmi_min is not None and args.bmi_max is not None
and args.bmi_max > args.bmi_min:
672         bmi_min, bmi_max = float(args.bmi_min), float(args.
bmi_max)
673     else:
674         bmi_min = float(np.nanmin(d0[col_bmi]))
675         bmi_max = float(np.nanmax(d0[col_bmi]))
676     b_grid = np.arange(bmi_min, bmi_max + 1e-9, args.bmi_step)
677
678     # 代表值 (中位数)
679     age_med = float(np.nanmedian(d0[col_age])) if (col_age and
col_age in d0.columns) else None
680     g_med = float(np.nanmedian(d0[col_g])) if (col_g and
col_g in d0.columns) else None

```

```

681     p_med = float(np.nanmedian(d0[col_p])) if (col_p and
        col_p in d0.columns) else None
682
683     print(f"[OK] 模型: {model_type}; sheet={sheet_name}; N={
        len(d0)}; "
684         f"GA∈[{d0[col_ga].min():.2f},{d0[col_ga].max():.2f}]
        ; "
685         f"BMI∈[{d0[col_bmi].min():.2f},{d0[col_bmi].max():.2
        f}]")
686     if model_type == 'MixedLM':
687         try:
688             print(f"[INFO] MixedLM 收敛: llf={getattr(result, '
        llf', np.nan):.3f}; scale≈{float(getattr(result, 'scale', np.
        nan)):.4f}")
689         except:
690             pass
691
692     # ----- 单次流程封装 -----
693     def run_pipeline_for_kappa(kappa_val: float, out_dir_use:
        str, fig_dir_use: str,
694                               draw_sensitivity_once: bool =
        True):
695         # 1)  $T^*(b)$ : 二分
696         T_star_loc = [tstar_for_bmi(result, model_type, design
        , b, args.q,
697                                   args.tmin, args.tmax, args
        .tstep, args.thr,
698                                   age_med, g_med, p_med,
699                                   var_mode=args.var_mode,
        kappa=kappa_val)
        for b in b_grid]
701         T_star_loc = np.asarray(T_star_loc, float)
702
703         # 2) 单调化 (Bayes 或 PAVA)
704         b_sorted_loc = np.asarray(b_grid)

```

```

705     T_bayes_lo = T_bayes_hi = None
706     if args.mono_method == 'bayes' and HAS_PYMC:
707         T_mean, T_lo, T_hi = bayes_monotone_curve(
708             b_sorted_loc, T_star_loc,
709             method=args.bayes_method, vi_steps=args.
vi_steps,
710             draws=args.draws, tune=args.tune, chains=args.
chains,
711             target_accept=args.target_accept, seed=args.
seed
712         )
713         T_mono_loc = T_mean
714         T_bayes_lo, T_bayes_hi = T_lo, T_hi
715     else:
716         if args.mono_method == 'bayes' and not HAS_PYMC:
717             warnings.warn("未检测到 PyMC, 自动回退 PAVA 单
调回归。", RuntimeWarning)
718         T_mono_loc = pava_monotone_curve(b_sorted_loc,
T_star_loc)
719
720     # 上包络
721     if args.envelope:
722         T_curve_loc = np.maximum(T_mono_loc, T_star_loc)
723         if T_bayes_lo is not None and T_bayes_hi is not
None:
724             T_bayes_lo = np.maximum(T_bayes_lo, T_star_loc
)
725             T_bayes_hi = np.maximum(T_bayes_hi, T_star_loc
)
726         else:
727             T_curve_loc = T_mono_loc
728
729     # 3) 台阶化
730     segs_loc = stepwise_segments_L2(b_sorted_loc,
T_curve_loc, K=args.K, min_width=args.min_width)

```

```

731         if args.mono_steps:
732             segs_loc = enforce_monotone_on_segments(segs_loc)
733
734             # ==== 新增：每段概率均值与风险指数（与第二问
一致） ====
735             def prob_fn_at(t_week: float, b_bmi: float) -> float:
736                 # 复用第三问同口径的概率函数（含 kappa、方差模型
等）
737                 return prob_ge_thr(result, model_type, design,
738                                     t_week, b_bmi, args.thr,
739                                     age_med, g_med, p_med,
740                                     var_mode=args.var_mode, kappa=
kappa_val)
741
742             rows_risk = compute_segment_metrics(
743                 b_grid=b_sorted_loc, segs=segs_loc, prob_fn_at=
prob_fn_at,
744                 alpha=getattr(args, 'alpha', 0.02), beta=getattr(
args, 'beta', 0.12),
745             )
746
747             import pandas as pd, os
748             df_risk = pd.DataFrame(rows_risk)
749             out_risk = os.path.join(out_dir_use, "
segments_q3_with_risk.csv")
750             df_risk.to_csv(out_risk, index=False, encoding="utf-8-
sig")
751             print(f"[OK@kappa={kappa_val:g}] 风险指数（分段）→ {
out_risk}")
752             # ==== 新增 END ====
753
754             # 4) 导出 CSV
755             os.makedirs(out_dir_use, exist_ok=True)
756             p1 = os.path.join(out_dir_use, 'q3_tstar_by_bmi.csv')
757             p2 = os.path.join(out_dir_use, 'q3_bmi_stepify_summary

```

```

.csv')
758     p3 = os.path.join(out_dir_use, '
q3_bmi_stepify_detailed.csv')
759     pd.DataFrame({"BMI": b_sorted_loc, "t_star":
T_star_loc, "t_curve": T_curve_loc})\
760         .to_csv(p1, index=False, encoding='utf-8-sig')
761     order = np.argsort(b_sorted_loc, kind="mergesort")
762     b_sorted2 = b_sorted_loc[order]; T_sorted_loc =
T_curve_loc[order]
763     summary_loc = summarize_segments(b_sorted2,
T_sorted_loc, segs_loc, roundto=args.roundto)
764     seg_id_loc = assign_segments(b_sorted2, segs_loc)
765     detailed_loc = pd.DataFrame({"BMI": b_sorted2, "t_star"
": T_sorted_loc, "seg_id": seg_id_loc})
766     summary_loc.to_csv(p2, index=False, encoding='utf-8-
sig')
767     detailed_loc.to_csv(p3, index=False, encoding='utf-8-
sig')
768
769     print(f"[OK@kappa={kappa_val:g}] t*(BMI) → {p1}")
770     print(f"[OK@kappa={kappa_val:g}] 台阶化 (汇总) → {p2}"
)
771     print(f"[OK@kappa={kappa_val:g}] 台阶化 (明细) → {p3}"
)
772     with pd.option_context('display.max_rows', 50, '
display.width', 120):
773         print(summary_loc)
774
775     # 5) 绘图
776     if args.plots:
777         fig_curve = os.path.join(fig_dir_use, '
fig_q3_curve.png')
778         fig_seg = os.path.join(fig_dir_use, '
fig_q3_segments.png')
779         fig_sens = os.path.join(fig_dir_use, '

```

```

fig_q3_sensitivity.png')
780     pathlib.Path(fig_dir_use).mkdir(parents=True,
exist_ok=True)

781
782     plot_curve(fig_curve, b_sorted_loc, T_star_loc,
T_curve_loc,
783                 dpi=args.fig_dpi, save_pdf=args.
save_pdf,
784                 cred_lo=(T_bayes_lo if (args.show_cred
and T_bayes_lo is not None) else None),
785                 cred_hi=(T_bayes_hi if (args.show_cred
and T_bayes_hi is not None) else None))
786     plot_segments(fig_seg, b_sorted2, T_sorted_loc,
segs_loc,
787                 dpi=args.fig_dpi, save_pdf=args.
save_pdf)

788
789     # 灵敏度 (只画一次)
790     if draw_sensitivity_once:
791         try:
792             kappas = [float(s) for s in str(args.
kappa_grid).split(',') if s.strip()!='']
793         except Exception:
794             kappas = [0.9, 1.0, 1.15, 1.25]
795
796     def _compute_T_curve_for_kappa(kappa_sens:
float):
797         T_star_k = [tstar_for_bmi(result,
model_type, design, b, args.q,
798                                     args.tmin, args.
tmax, args.tstep, args.thr,
799                                     age_med, g_med,
p_med,
800                                     var_mode=args.
var_mode, kappa=kappa_sens)

```

```

801         for b in b_sorted_loc]
802         T_star_k = np.asarray(T_star_k, float)
803         if args.mono_method == 'bayes' and
HAS_PYMC:
804             T_mean_k, _, _ = bayes_monotone_curve(
805                 b_sorted_loc, T_star_k,
806                 method=args.bayes_method, vi_steps
=args.vi_steps,
807                 draws=args.draws, tune=args.tune,
chains=args.chains,
808                 target_accept=args.target_accept,
seed=args.seed
809             )
810             T_mono_k = T_mean_k
811         else:
812             T_mono_k = pava_monotone_curve(
b_sorted_loc, T_star_k)
813             T_curve_k = np.maximum(T_mono_k, T_star_k)
814             if args.envelope else T_mono_k
815             return b_sorted_loc, T_curve_k
816
817             plot_sensitivity(fig_sens, kappas,
_compute_T_curve_for_kappa,
818             dpi=args.fig_dpi, save_pdf=
args.save_pdf)
819             print(f"[OK@kappa={kappa_val:g}] 图已保存: \n
- {fig_curve}\n - {fig_seg}\n - {fig_sens}")
820         else:
821             print(f"[OK@kappa={kappa_val:g}] 图已保存: \n
- {fig_curve}\n - {fig_seg}")
822
823         # 6) 方案A: 按 G 分层 (仅当提供 --g-grid 且模型含G且未
omit_g 时执行)
824         if (args.g_grid is not None) and (('G' in design.get('
age_gp_cols', [])) and (not args.omit_g)):

```

```

824         try:
825             g_list = [float(s) for s in str(args.g_grid).
split(',') if s.strip()!='']
826         except Exception:
827             g_list = []
828
829         if g_list:
830             curves_by_g = []
831             df_out = pd.DataFrame({'BMI': b_sorted_loc})
832             for gv in g_list:
833                 # 重新计算该 G 的 t* 与单调化
834                 T_star_g = [tstar_for_bmi(result,
model_type, design, b, args.q,
835                                     args.tmin, args.
tmax, args.tstep, args.thr,
836                                     age_med, gv,
p_med,
837                                     var_mode=args.
var_mode, kappa=kappa_val)
838                             for b in b_sorted_loc]
839                 T_star_g = np.asarray(T_star_g, float)
840                 # 单调化
841                 if args.mono_method == 'bayes' and
HAS_PYMC:
842                     T_mean_g, _, _ = bayes_monotone_curve(
843                         b_sorted_loc, T_star_g,
844                         method=args.bayes_method, vi_steps
=args.vi_steps,
845                         draws=args.draws, tune=args.tune,
chains=args.chains,
846                         target_accept=args.target_accept,
seed=args.seed
847                     )
848                     T_mono_g = T_mean_g
849                 else:

```

```

850         T_mono_g = pava_monotone_curve(
b_sorted_loc, T_star_g)
851         T_curve_g = np.maximum(T_mono_g, T_star_g)
if args.envelope else T_mono_g
852
853         curves_by_g.append((gv, T_curve_g))
854         df_out[f't_curve_G{int(gv)}'] = T_curve_g
855
856         # 导出 CSV + 两张图
857         p_csv_g = os.path.join(out_dir_use, '
q3_tstar_by_bmi_vs_g.csv')
858         p_fig_g = os.path.join(fig_dir_use, '
fig_q3_curves_vs_g.png')
859         p_fig_dlt = os.path.join(fig_dir_use, '
fig_q3_delta_tstar_vs_g.png')
860         df_out.to_csv(p_csv_g, index=False, encoding='
utf-8-sig')
861         plot_curves_vs_g(p_fig_g, b_sorted_loc,
curves_by_g, dpi=args.fig_dpi, save_pdf=args.save_pdf)
862         plot_delta_tstar_vs_g(p_fig_dlt, b_sorted_loc,
curves_by_g, dpi=args.fig_dpi, save_pdf=args.save_pdf)
863
864         print(f"[OK@kappa={kappa_val:g}] 按 G 分层的曲
线/差值图已生成: \n - {p_fig_g}\n - {p_fig_dlt}\n - {p_csv_g
}")
865     else:
866         print("[WARN] --g-grid 解析为空, 已跳过按 G 分
层。")
867     elif args.g_grid is not None and (args.omit_g or ('G'
not in design.get('age_gp_cols', []))):
868         print("[WARN] 当前模型未包含 G, 已忽略 --g-grid
(按 G 分层没有意义)。")
869
870     # ----- 批量或单次执行 -----
871     if args.kappa_batch:

```

```

872     try:
873         k_list = [float(s) for s in args.kappa_batch.split
874                   (' ','') if s.strip()!='']
875     except Exception:
876         raise ValueError("解析 --kappa-batch 失败, 请用英文逗号分隔, 例如 1.00,1.15,1.25")
877
878     print(f"[INFO] 批量运行 kappa = {k_list}")
879     for idx, kappa_val in enumerate(k_list):
880         if args.batch_subdir:
881             out_use = os.path.join(args.outdir, f"kappa_{kappa_val:.2f}")
882             fig_use = out_use
883         else:
884             suffix = f"_k{kappa_val:g}".replace('.', 'p')
885             out_use = f"{args.outdir}{suffix}"
886             fig_use = args.figdir or out_use
887             run_pipeline_for_kappa(kappa_val, out_use, fig_use
888                                   ,
889                                   draw_sensitivity_once=(idx
890                                                         == 0))
891         else:
892             run_pipeline_for_kappa(args.kappa, args.outdir, args.
893                                   figdir or args.outdir,
894                                   draw_sensitivity_once=True)
895
896 if __name__ == '__main__':
897     warnings.filterwarnings("ignore", category=RuntimeWarning)
898     warnings.filterwarnings("ignore", category=FutureWarning)
899     main()

```

q4.py

```

1 # -*- coding: utf-8 -*-
2 """

```

```

3 Q4 Transfer++
4 - Per-chrom empirical-null (normal / Student-t), winsorization
  , batch covariate
5 - Grid-scan for FPR control
6 - Per-chromosome thresholds (z/q/consistency)
7 - Chinese-friendly & polished visualizations (+ multi-page PDF
  report)
8
9 Typical run:
10 python q4_transfer.py --xlsx "附件.xlsx" --male-sheet "男胎
   检测数据" --female-sheet "女胎检测数据" \
11   --col-subject "孕妇代码" --col-ga "检测孕周" --col-bmi "孕
   妇BMI" \
12   --col-z13 "13号染色体的Z值" --col-z18 "18号染色体的Z值" --
   col-z21 "21号染色体的Z值" \
13   --qc-features "GA,BMI,reads,align_rate,dup_rate,
   unique_reads,gc" \
14   --batch-col "检测日期" --student-chrom "21" --winsor 0.01
   \
15   --grid-scan --target-fpr 0.01 --grid-z "2.6,2.8,3.0,3.2"
   --grid-q "0.03,0.05,0.08" --grid-consist "2,3" \
16   --outdir "out_q4_transfer_final"
17
18 To set per-chromosome thresholds (optional, override global):
19   --z-thr-21 2.6 --q-thr-21 0.05 --consist-21 2
20   ""
21
22 import os, re, json, argparse, warnings
23 from typing import List, Tuple, Dict, Optional
24
25 import numpy as np
26 import pandas as pd
27 import matplotlib.pyplot as plt
28 from matplotlib import font_manager as _fm
29 from matplotlib.backends.backend_pdf import PdfPages

```

```

30 from math import erf
31
32 # Optional SciPy
33 try:
34     import scipy.stats as sps
35 except Exception:
36     sps = None
37
38 # ----- Pretty & Chinese Fonts
39 _PRETTY_PALETTE = ["#4C78A8", "#F58518", "#E45756", "#72B7B2",
40                   "#54A24B",
41                   "#EECA3B", "#B279A2", "#FF9DA7", "#9D755D",
42                   "#BAB0AC"]
43
44 def _pick_chinese_font():
45     candidates = ["Microsoft YaHei", "Microsoft YaHei UI", "微
46                 软雅黑",
47                 "SimHei", "黑体", "PingFang SC", "苹方-简",
48                 "Noto Sans CJK SC", "Source Han Sans SC", "
49                 思源黑体",
50                 "WenQuanYi Micro Hei"]
51     available = {f.name for f in _fm.fontManager.ttflist}
52     for name in candidates:
53         if name in available:
54             return name
55     return None
56
57 def _setup_mpl():
58     import matplotlib as mpl
59     font = _pick_chinese_font()
60     if font:
61         mpl.rcParams["font.sans-serif"] = [font, "DejaVu Sans",
62         "Arial"]
63     mpl.rcParams["axes.unicode_minus"] = False

```

```

59     mpl.rcParams["axes.titleweight"] = "bold"
60     mpl.rcParams["figure.dpi"] = 140
61     mpl.rcParams["axes.grid"] = True
62     mpl.rcParams["grid.alpha"] = 0.25
63
64     def _annotate_bars(ax):
65         for container in ax.containers:
66             ax.bar_label(container, fmt="%.0f", padding=2,
67                           fontsize=9)
68
69     # ----- Math & Stats
70     -----
71     def _norm_cdf(x: np.ndarray) -> np.ndarray:
72         x = np.asarray(x, dtype=float)
73         return 0.5 * (1.0 + np.vectorize(erf)(x / np.sqrt(2.0)))
74
75     def two_tailed_p_from(z: np.ndarray, family: str = "normal",
76                           df: Optional[float] = None) -> np.ndarray:
77         z = np.asarray(z, dtype=float)
78         if family == "student" and sps is not None and df is not
79             None and df > 1:
80             return 2.0 * (1.0 - sps.t.cdf(np.abs(z), df=df))
81         return 2.0 * (1.0 - _norm_cdf(np.abs(z)))
82
83     def bh_fdr(p: np.ndarray) -> np.ndarray:
84         p = np.asarray(p, dtype=float)
85         n = p.size
86         if n == 0: return p
87         order = np.argsort(p)
88         ranked = p[order]
89         q = np.empty_like(ranked)
90         prev = 1.0
91         for i in range(n, 0, -1):
92             val = ranked[i-1] * n / i
93             prev = min(prev, val)

```

```

90     q[i-1] = prev
91     out = np.empty_like(q)
92     out[order] = np.clip(q, 0, 1)
93     return out
94
95 def parse_ga_bins(spec: str) -> List[Tuple[str, float, float
96 ]]:
97     bins = []
98     parts = [p.strip() for p in spec.split(",") if p.strip()]
99     for p in parts:
100         name, rng = [x.strip() for x in p.split(":", 1)]
101         if rng.startswith("<="):
102             upper = float(rng[2:]); bins.append((name, -np.inf
103 , upper))
104         elif rng.startswith(">="):
105             lower = float(rng[2:]); bins.append((name, lower,
106 np.inf))
107         elif rng.startswith(">"):
108             lower = float(rng[1:]); bins.append((name, np.
109 nextafter(lower, np.inf), np.inf))
110         elif rng.startswith("<"):
111             upper = float(rng[1:]); bins.append((name, -np.inf
112 , np.nextafter(upper, -np.inf)))
113         else:
114             if "-" in rng:
115                 a, b = rng.split("-", 1); bins.append((name,
116 float(a), float(b)))
117             else:
118                 v = float(rng); bins.append((name, v, v))
119     return bins
120
121 def assign_bins(values: pd.Series, bins: List[Tuple[str, float
122 , float]], cat_name="ga_bin") -> pd.Series:
123     labels = []
124     for v in values.astype(float).values:

```

```

118         label = None
119         for name, lo, hi in bins:
120             if (v >= lo) and (v <= hi):
121                 label = name; break
122         labels.append(label)
123     return pd.Series(labels, index=values.index, name=cat_name
124 )
125
126 def robust_loc_scale(x: np.ndarray, robust: bool = True) ->
127     Tuple[float, float]:
128     x = np.asarray(x, dtype=float)
129     if x.size == 0: return 0.0, 1.0
130     if robust:
131         med = float(np.median(x))
132         mad = float(np.median(np.abs(x - med)))
133         sigma = 1.4826 * mad
134         if sigma <= 1e-12:
135             sigma = float(np.std(x, ddof=1) if x.size > 1 else
136 1.0)
137         return med, (sigma if sigma > 0 else 1.0)
138     mu = float(np.mean(x))
139     sd = float(np.std(x, ddof=1) if x.size > 1 else 1.0)
140     return mu, (sd if sd > 0 else 1.0)
141
142 def winsorize(x: np.ndarray, p: float = 0.01) -> np.ndarray:
143     if p <= 0: return x
144     lo, hi = np.quantile(x, [p, 1-p])
145     return np.clip(x, lo, hi)
146
147 def fit_ols(X: np.ndarray, y: np.ndarray) -> np.ndarray:
148     XtX = X.T @ X
149     ridge = 1e-8 * np.eye(X.shape[1])
150     beta = np.linalg.solve(XtX + ridge, X.T @ y)
151     return beta

```

```

150 def design_matrix(df: pd.DataFrame, features: List[str]) -> np
    .ndarray:
151     cols = []
152     for f in features:
153         if f == "intercept":
154             cols.append(np.ones((len(df), 1)))
155         else:
156             if f not in df.columns:
157                 warnings.warn(f"[WARN] 缺少特征列 '{f}', 用 0
填充。")
158                 cols.append(np.zeros((len(df), 1)))
159             else:
160                 cols.append(df[[f]].astype(float).values)
161     return np.hstack(cols) if cols else np.ones((len(df), 1))
162
163 def find_first_existing(df: pd.DataFrame, candidates: List[str]
    ], default: Optional[str] = None) -> Optional[str]:
164     for c in candidates:
165         if c in df.columns: return c
166     return default
167
168 def load_sheets(xlsx_path: str, male_sheet: Optional[str],
    female_sheet: Optional[str]) -> Tuple[pd.DataFrame, pd.
    DataFrame]:
169     xls = pd.ExcelFile(xlsx_path); sheets = xls.sheet_names
170     def _read(sname): return pd.read_excel(xlsx_path,
    sheet_name=sname)
171     mdf = _read(male_sheet) if (male_sheet and male_sheet in
    sheets) else None
172     fdf = _read(female_sheet) if (female_sheet and
    female_sheet in sheets) else None
173     if mdf is None:
174         for s in sheets:
175             if "男" in s: mdf = _read(s); break
176     if fdf is None:

```

```

177         for s in sheets:
178             if "女" in s: fdf = _read(s); break
179     if mdf is None or fdf is None:
180         for s in sheets:
181             df = _read(s)
182             sex_col = find_first_existing(df, ["性别", "胎儿性别", "sex", "Sex"], None)
183             if sex_col is not None:
184                 mdf = df[df[sex_col].astype(str).str.contains(
185                     "男|male|Male", case=False, regex=True)].copy()
186                 fdf = df[df[sex_col].astype(str).str.contains(
187                     "女|female|Female", case=False, regex=True)].copy()
188                 break
189             if mdf is None or fdf is None:
190                 raise ValueError("未能找到男胎/女胎数据表。请通过 --
191                 male-sheet 与 --female-sheet 指定。")
192             return mdf, fdf
193
194 # ----- GA parsing & numeric cleaning -----
195 def _parse_ga_value(val):
196     if val is None or (isinstance(val, float) and np.isnan(val)):
197         return np.nan
198     if isinstance(val, (int, float)) and not isinstance(val,
199         bool): return float(val)
200     s = str(val).strip()
201     if s == "": return np.nan
202     s = s.replace("周数", "w").replace("孕周", "w").replace("周",
203         "w").replace("天", "d").replace("W", "w").replace("D", "d")
204     s = re.sub(r"\s+", "", s)
205     m = re.match(r"^(?d+)\s*w\+?(?d+)\s*d?$", s)
206     if m:
207         w = int(m.group(1)); d = int(m.group(2)); w += d//7; d
208         %= 7; return float(w) + d/7.0
209     m = re.match(r"^(?d+)\s*w$", s)
210     if m: return float(int(m.group(1)))

```

```

204     m = re.match(r"^(\d+)\s+(\d+)$", s)
205     if m:
206         w = int(m.group(1)); d = int(m.group(2)); w += d//7; d
207         %= 7; return float(w) + d/7.0
208     m = re.match(r"^(\d+)\s*w(\d+)\s*d$", s)
209     if m:
210         w = int(m.group(1)); d = int(m.group(2)); w += d//7; d
211         %= 7; return float(w) + d/7.0
212     m = re.match(r"^(\d+)\s*d$", s)
213     if m: return int(m.group(1))/7.0
214     m = re.match(r"^(\d+(?:\.\d+)?)$", s)
215     if m: return float(m.group(1))
216     return np.nan
217
218 def coerce_ga_weeks(series: pd.Series) -> pd.Series:
219     parsed = series.apply(_parse_ga_value)
220     n_total = len(series); n_nan = int(parsed.isna().sum())
221     if n_nan > 0:
222         examples = series[parsed.isna()].astype(str).head(5).
223         tolist()
224         warnings.warn(f"[WARN] 有 {n_nan}/{n_total} 条孕周无法
225         解析, 例如: {examples}")
226     return parsed
227
228 def _coerce_num_series(s: pd.Series) -> pd.Series:
229     if s.dtype == object:
230         s2 = s.astype(str).str.replace('%', '', regex=False).str
231         .replace(',', '', regex=False).str.strip()
232         s2 = pd.to_numeric(s2, errors='coerce')
233     else:
234         s2 = pd.to_numeric(s, errors='coerce')
235     if s2.notna().sum() > 0 and ((s2.dropna() > 1).mean() > 0.5)
236     and (s2.max() <= 100):
237         s2 = s2/100.0
238     return s2

```

```

233
234 # ----- Batch covariate -----
235 def add_batch_ohe(df: pd.DataFrame, col: str, max_levels: int
    = 20) -> Tuple[pd.DataFrame, List[str]]:
236     if (col is None) or (col not in df.columns): return df, []
237     ser = df[col]
238     try:
239         dt = pd.to_datetime(ser, errors="coerce")
240     except Exception:
241         dt = None
242     if dt is not None and dt.notna().any():
243         base = dt.min()
244         df["_batch_days"] = (dt - base).dt.days.astype(float)
245         return df, ["_batch_days"]
246     vals = ser.astype(str)
247     top = vals.value_counts().index[:max_levels]
248     ohe = pd.get_dummies(vals.where(vals.isin(top), other="
    _other"), prefix="batch", drop_first=True)
249     df = pd.concat([df, ohe], axis=1)
250     return df, list(ohe.columns)
251
252 # ----- Core calibration & application -----
253 def calibrate_and_apply(
254     male: pd.DataFrame,
255     female: pd.DataFrame,
256     chrom_cols: Dict[str, str],
257     ga_col: str,
258     bmi_col: Optional[str],
259     subject_col: str,
260     qc_features: List[str],
261     ga_bins_spec: str,
262     outdir: str,
263     robust_scale: bool = True,
264     min_n_bin: int = 25,
265     winsor_p: float = 0.0,

```

```

266     null_family_per_chrom: Dict[str, str] = None,
267     student_df_per_chrom: Dict[str, Optional[float]] = None,
268     alpha: float = 0.05,
269     z_thr: float = 3.0,
270     q_thr: float = 0.05,
271     suspect_z: float = 2.5,
272     suspect_q: float = 0.10,
273     min_consistency: int = 2,
274     # Per-chrom overrides (optional)
275     z_thr_map: Optional[Dict[str, float]] = None,
276     q_thr_map: Optional[Dict[str, float]] = None,
277     min_consistency_map: Optional[Dict[str, int]] = None,
278     estimate_fpr_with_male: bool = False,
279 ):
280     os.makedirs(outdir, exist_ok=True)
281     _setup_mpl()
282
283     bins = parse_ga_bins(ga_bins_spec)
284     male = male.copy(); female = female.copy()
285
286     if ga_col not in male.columns or ga_col not in female.
columns:
287         raise ValueError(f"孕周列 '{ga_col}' 在数据表中不存
在。")
288
289     male["ga_bin"] = assign_bins(male[ga_col], bins, "ga_bin")
290     female["ga_bin"] = assign_bins(female[ga_col], bins, "
ga_bin")
291
292     feat_list = ["intercept"] + [f for f in qc_features if f
and f != "intercept"]
293
294     cal_rows = []
295     female_rows = []
296

```

```

297     for chrom, colname in chrom_cols.items():
298         if colname not in male.columns or colname not in
female.columns:
299             warnings.warn(f"[WARN] 缺少染色体列 '{colname}',
跳过 {chrom}."); continue
300             family = (null_family_per_chrom or {}).get(chrom, "
normal")
301             df_override = (student_df_per_chrom or {}).get(chrom,
None)
302
303             for (bin_name, _, _) in bins:
304                 msub = male[male["ga_bin"] == bin_name].copy()
305                 fsub = female[female["ga_bin"] == bin_name].copy()
306                 if msub.shape[0] < min_n_bin:
307                     warnings.warn(f"[WARN] 男胎 {chrom}/{bin_name}
样本数 {msub.shape[0]} < {min_n_bin}, 使用全体男胎代替。")
308                     msub = male.copy()
309
310                 X_m = design_matrix(msub, feat_list)
311                 y_m = msub[colname].astype(float).values
312                 beta = fit_ols(X_m, y_m)
313                 resid_m = y_m - X_m @ beta
314                 if winsor_p > 0:
315                     resid_m = winsorize(resid_m, winsor_p)
316                 mu0, sigma0 = robust_loc_scale(resid_m, robust=
robust_scale)
317
318                 # Estimate df for t family if needed
319                 df_est = None
320                 if family == "student":
321                     if df_override and df_override > 1:
322                         df_est = float(df_override)
323                     elif sps is not None:
324                         try:
325                             df_fit, _, _ = sps.t.fit(resid_m)

```

```

326         if df_fit and df_fit > 1: df_est =
float(df_fit)
327     except Exception:
328         df_est = None
329     else:
330         m4 = np.mean((resid_m - np.mean(resid_m))
**4)
331         v = np.var(resid_m); g2 = m4 / (v**2 + 1e
-12) - 3.0
332         if g2 > 0: df_est = 6.0 / g2 + 4.0
333
334     cal_rows.append({
335         "chrom": chrom,
336         "ga_bin": bin_name,
337         "n_male_used": int(len(msub)),
338         "features": ",".join(feet_list),
339         "beta_json": json.dumps(beta.tolist()),
340         "mu0": float(mu0),
341         "sigma0": float(sigma0),
342         "null_family": family,
343         "student_df": df_est
344     })
345
346     if fsub.shape[0] == 0: continue
347     X_f = design_matrix(fsub, feat_list)
348     y_f = fsub[colname].astype(float).values
349     resid_f = y_f - X_f @ beta
350     ztilde = (resid_f - mu0) / (sigma0 if sigma0>0
else 1.0)
351     pvals = two_tailed_p_from(ztilde, family=family,
df=df_est)
352
353     female_rows.append(pd.DataFrame({
354         "subject_id": fsub[subject_col].astype(str).
values if subject_col in fsub.columns else np.arange(len(

```

```

fsub)).astype(str),
355         "chrom": chrom,
356         "ga_bin": bin_name,
357         "GA_weeks": fsub[ga_col].astype(float).values,
358         "BMI": fsub[bmi_col].astype(float).values if (
bmi_col and bmi_col in fsub.columns) else np.nan,
359         "y": y_f,
360         "y_pred": (X_f @ beta),
361         "resid": resid_f,
362         "z": ztilde,
363         "z_abs": np.abs(ztilde),
364         "p": pvals
365     )))
366
367     cal_df = pd.DataFrame(cal_rows)
368     cal_path = os.path.join(outdir, "male_calibration_summary.
csv")
369     cal_df.to_csv(cal_path, index=False)
370
371     if len(female_rows)==0: raise ValueError("未生成女胎结果
(检查列名/数据是否匹配)。")
372     fem = pd.concat(female_rows, ignore_index=True)
373
374     # q-values per (chrom, ga_bin)
375     fem["q"] = np.nan
376     for (chrom, bin_name), g in fem.groupby(["chrom", "ga_bin"
]):
377         idx = g.index.values; fem.loc[idx, "q"] = bh_fdr(g["p"
].values)
378
379     # ----- Sample-level labeling with per-chrom thresholds
-----
380     def get_z_thr(c): return (z_thr_map or {}).get(c, z_thr)
381     def get_q_thr(c): return (q_thr_map or {}).get(c, q_thr)
382

```

```

383 fem["z_thr_used"] = fem["chrom"].map(get_z_thr)
384 fem["q_thr_used"] = fem["chrom"].map(get_q_thr)
385
386 def label_row(z_abs, q, zt, qt):
387     if (z_abs >= zt) and (q <= qt): return "阳性"
388     if (z_abs >= suspect_z) or (q <= suspect_q): return "
可疑(建议复检)"
389     return "阴性"
390
391 fem["label_sample"] = [label_row(a, b, c, d) for a, b, c,
d in zip(
392     fem["z_abs"].values, fem["q"].values, fem["z_thr_used"
].values, fem["q_thr_used"].values
393 )]
394 fem_path = os.path.join(outdir, "female_anomaly_results.
csv"); fem.to_csv(fem_path, index=False)
395
396 # ----- Subject-level aggregation (per-chrom consistency
) -----
397 def get_k(c): return (min_consistency_map or {}).get(c,
min_consistency)
398
399 subj_rows = []
400 for sid, gsub in fem.groupby("subject_id"):
401     for chrom, gc in gsub.groupby("chrom"):
402         zt = get_z_thr(chrom); qt = get_q_thr(chrom); k =
int(get_k(chrom))
403         pos_hits = gc[(gc["z_abs"]>=zt) & (gc["q"]<=qt)]
404         pos_sgn = np.sign(gc.loc[gc["z_abs"]>=zt, "z"].
values)
405         maj = 0 if pos_sgn.size==0 else int(np.sign(np.sum
(pos_sgn)))
406         consistent = (pos_hits.shape[0] >= k) and (np.sum(
np.sign(pos_hits["z"].values) == maj) >= k)
407         if consistent and pos_hits.shape[0] >= k:

```

```

408         lab = "阳性"; t_earliest = float(pos_hits["
GA_weeks"].min())
409         elif (gc["z_abs"].ge(zt).any()) or (gc["q"].le(
suspect_q).any()):
410             lab = "可疑(建议复检)"; ridx = int(np.argmax(
gc["z_abs"].values)); t_earliest = float(gc["GA_weeks"].
values[ridx])
411         else:
412             lab = "阴性"; t_earliest = np.nan
413             subj_rows.append({"subject_id": sid, "chrom":
chrom, "label_subject": lab, "earliest_weeks": t_earliest,
414                             "n_samples": int(gc.shape[0]), "
n_pos_hits": int(pos_hits.shape[0]),
415                             "z_thr_used": zt, "q_thr_used":
qt, "consistency_used": k})
416             subj = pd.DataFrame(subj_rows); subj_path = os.path.join(
outdir, "subject_summary.csv"); subj.to_csv(subj_path, index
=False)
417
418             # ----- Optional quick male FPR estimate (at current
thresholds) -----
419             fpr_val = None
420             if estimate_fpr_with_male:
421                 mrows = []
422                 for _, row in cal_df.iterrows():
423                     chrom = row["chrom"]; family = row["null_family"];
df_est = row["student_df"]
424                     msub = male[male["ga_bin"] == row["ga_bin"]].copy
()
425                     if msub.empty: continue
426                     feat = row["features"].split(","); beta = np.array
(json.loads(row["beta_json"]))
427                     X_m = design_matrix(msub, feat); y_m = msub[
chrom_cols[chrom]].astype(float).values
428                     resid_m = y_m - X_m @ beta

```

```

429         ztilde_m = (resid_m - row["mu0"]) / (row["sigma0"]
430         if row["sigma0"]>0 else 1.0)
431         p_m = two_tailed_p_from(ztilde_m, family=family,
432         df=df_est)
433         mrows.append(pd.DataFrame({
434             "subject_id": msub[subject_col].astype(str).
435             values if subject_col in msub.columns else np.arange(len(
436             msub)).astype(str),
437             "chrom": chrom, "ga_bin": row["ga_bin"], "
438             z_abs": np.abs(ztilde_m), "p": p_m
439             }))
440         if mrows:
441             mal = pd.concat(mrows, ignore_index=True)
442             mal["q"] = np.nan
443             for (chrom, bin_name), g in mal.groupby(["chrom",
444             "ga_bin"]):
445                 mal.loc[g.index, "q"] = bh_fdr(g["p"].values)
446                 msubj_rows = []
447                 for sid, gsub in mal.groupby("subject_id"):
448                     for chrom, gc in gsub.groupby("chrom"):
449                         zt = get_z_thr(chrom); qt = get_q_thr(
450                         chrom); k = int(get_k(chrom))
451                         pos_hits = gc[(gc["z_abs"]>=zt) & (gc["q"
452                         ]<=qt)]
453                         consistent = (pos_hits.shape[0] >= k)
454                         lab = "阳性" if consistent else ("可疑(建
455                         议复检)" if ((gc["z_abs"].ge(zt).any()) or (gc["q"].le(qt).
456                         any())) else "阴性")
457                         msubj_rows.append({"subject_id": sid, "
458                         chrom": chrom, "label_subject": lab})
459                         msubj = pd.DataFrame(msubj_rows)
460                         if not msubj.empty:
461                             fpr_val = float((msubj["label_subject"]=="阳性
462                             ").mean())
463

```

```

452 # ----- Visualizations
453 pdf_path = os.path.join(outdir, "figures_report.pdf")
454 with PdfPages(pdf_path) as pdf:
455     # 1) Subject counts by chromosome
456     counts = subj.groupby(["chrom", "label_subject"]).size
457     ().unstack(fill_value=0)
458     ax = counts.plot(kind="bar", rot=0)
459     for i, container in enumerate(ax.containers):
460         color = _PRETTY_PALETTE[i % len(_PRETTY_PALETTE)]
461         for bar in container:
462             bar.set_color(color); bar.set_edgecolor("white")
463     _annotate_bars(ax)
464     ax.set_title("各染色体的个体级结论分布")
465     ax.set_xlabel("染色体"); ax.set_ylabel("人数")
466     plt.tight_layout(); plt.savefig(os.path.join(outdir, "chart_subject_label_counts.png"), bbox_inches="tight"); pdf.savefig(); plt.close()
467
468     # 2) |z| histogram (global) + vertical line at global z_thr
469     plt.figure()
470     fem["z_abs"].plot(kind="hist", bins=40)
471     plt.axvline(x=z_thr, linestyle="--", linewidth=1.5, label=f"全局阈值 z={z_thr}")
472     plt.legend()
473     plt.title("女胎样本 |z| 分布"); plt.xlabel("|z|"); plt.ylabel("频数")
474     plt.tight_layout(); plt.savefig(os.path.join(outdir, "chart_zabs_hist.png"), bbox_inches="tight"); pdf.savefig(); plt.close()
475
476     # 3) Earliest weeks boxplot
477     dfw = subj[subj["label_subject"].isin(["阳性", "可疑(建

```

```

议复检)"])]
477     if not dfw.empty and "earliest_weeks" in dfw.columns:
478         data = [dfw[dfw["label_subject"]=="阳性"]["
earliest_weeks"].dropna().values,
479                 dfw[dfw["label_subject"]=="可疑(建议复检)"
][ "earliest_weeks"].dropna().values]
480         labels = ["阳性", "可疑"]
481         plt.figure()
482         bp = plt.boxplot(data, tick_labels=labels,
patch_artist=True)
483         for i, box in enumerate(bp["boxes"]):
484             box.set_facecolor(_PRETTY_PALETTE[i % len(
_Pretty_Palette)]); box.set_edgecolor("black")
485             plt.title("首次达到阈值的孕周 (按最终结论)"); plt
.ylabel("孕周 (周)")
486             plt.tight_layout(); plt.savefig(os.path.join(
outdir, "chart_earliest_weeks_box.png"), bbox_inches="tight")
; pdf.savefig(); plt.close()

487
488     # 4) GA vs |z| scatter per chromosome
489     for chrom in sorted(fem["chrom"].unique(), key=lambda
x: int(x)):
490         sub = fem[fem["chrom"]==chrom].copy()
491         if sub.empty: continue
492         plt.figure()
493         colors = {"阳性": _PRETTY_PALETTE[2], "可疑(建议复
检)": _PRETTY_PALETTE[1], "阴性": _PRETTY_PALETTE[0]}
494         for lab, dfc in sub.groupby("label_sample"):
495             plt.scatter(dfc["GA_weeks"], dfc["z_abs"], s
=18, alpha=0.85, label=str(lab), c=colors.get(lab,
_Pretty_Palette[0]))
496             # per-chrom threshold line
497             plt.axhline(y=get_z_thr(chrom), linestyle="--",
linewidth=1.5, label=f"z 阈值({chrom})={get_z_thr(chrom)}")
498             plt.xlabel("孕周 (周)"); plt.ylabel("|z|")

```

```

499         plt.title(f"孕周 vs |z| ({chrom} 号染色体)")
500         plt.legend()
501         plt.tight_layout()
502         fn = os.path.join(outdir, f"chart_ga_vs_z_chr{
chrom}.png")
503         plt.savefig(fn, bbox_inches="tight"); pdf.savefig(
()); plt.close()
504
505         # ----- Meta JSON -----
506         meta = {"z_thr": z_thr, "q_thr": q_thr, "suspect_z":
suspect_z, "suspect_q": suspect_q,
507               "min_consistency": min_consistency, "winsor_p":
winsor_p,
508               "z_thr_map": z_thr_map, "q_thr_map": q_thr_map, "
min_consistency_map": min_consistency_map,
509               "null_family_per_chrom": null_family_per_chrom, "
estimated_FPR_male": fpr_val}
510         with open(os.path.join(outdir, "run_meta_plus.json"), "w",
encoding="utf-8") as f:
511             json.dump(meta, f, ensure_ascii=False, indent=2)
512
513         return fem, subj, cal_df, fpr_val
514
515         # ----- Grid scan -----
516     def grid_scan_thresholds(male: pd.DataFrame, fem: pd.DataFrame
, cal_df: pd.DataFrame,
517                             chrom_cols: Dict[str, str],
518                             subject_col: str, ga_col: str,
519                             z_grid: List[float], q_grid: List[
float], consist_grid: List[int],
520                             null_family_per_chrom: Dict[str, str])
-> pd.DataFrame:
521         rows = []
522         mrows = []
523         for _, row in cal_df.iterrows():

```

```

523     chrom = row["chrom"]; family = row["null_family"];
df_est = row["student_df"]
524     msub = male[male["ga_bin"] == row["ga_bin"]].copy()
525     if msub.empty: continue
526     feat = row["features"].split(","); beta = np.array(
json.loads(row["beta_json"]))
527     X_m = design_matrix(msub, feat); y_m = msub[chrom_cols
[chrom]].astype(float).values
528     resid_m = y_m - X_m @ beta
529     ztilde_m = (resid_m - row["mu0"]) / (row["sigma0"] if
row["sigma0"]>0 else 1.0)
530     p_m = two_tailed_p_from(ztilde_m, family=family, df=
df_est)
531     mrows.append(pd.DataFrame({
532         "subject_id": msub[subject_col].astype(str).values
if subject_col in msub.columns else np.arange(len(msub)).
astype(str),
533         "chrom": chrom, "ga_bin": row["ga_bin"], "z_abs":
np.abs(ztilde_m), "p": p_m
534     }))
535     if not mrows:
536         warnings.warn("男胎数据不足，无法做阈值网格扫描。");
return pd.DataFrame()
537     mal = pd.concat(mrows, ignore_index=True)
538     mal["q"] = np.nan
539     for (chrom, bin_name), g in mal.groupby(["chrom", "ga_bin"
]):
540         mal.loc[g.index, "q"] = bh_fdr(g["p"].values)
541
542     for zt in z_grid:
543         for qt in q_grid:
544             for k in consist_grid:
545                 msubj_rows = []
546                 for sid, gsub in mal.groupby("subject_id"):
547                     for chrom, gc in gsub.groupby("chrom"):

```

```

548         pos_hits = gc[(gc["z_abs"]>=zt) & (gc[
"q"]<=qt)]
549         consistent = (pos_hits.shape[0] >= k)
550         lab = "阳性" if consistent else ("可疑
" if ((gc["z_abs"].ge(zt).any()) or (gc["q"].le(qt).any()))
else "阴性")
551         msubj_rows.append({"subject_id": sid,
"chrom": chrom, "label": lab})
552         msubj = pd.DataFrame(msubj_rows)
553         fpr = float((msubj["label"]=="阳性").mean())
if not msubj.empty else np.nan
554         fsubj_rows = []
555         for sid, gsub in fem.groupby("subject_id"):
556             for chrom, gc in gsub.groupby("chrom"):
557                 pos_hits = gc[(gc["z_abs"]>=zt) & (gc[
"q"]<=qt)]
558                 sgn = np.sign(gc.loc[gc["z_abs"]>=zt,
"z"].values)
559                 maj = 0 if sgn.size==0 else int(np.
sign(np.sum(sgn)))
560                 consistent = (pos_hits.shape[0] >= k)
and (np.sum(np.sign(pos_hits["z"].values)==maj) >= k)
561                 lab = "阳性" if consistent else ("可疑
" if ((gc["z_abs"].ge(zt).any()) or (gc["q"].le(qt).any()))
else "阴性")
562                 fsubj_rows.append({"subject_id": sid,
"chrom": chrom, "label": lab})
563                 fsubj = pd.DataFrame(fsubj_rows)
564                 pos_cnt = int((fsubj["label"]=="阳性").sum())
if not fsubj.empty else 0
565                 rows.append({"z_thr": zt, "q_thr": qt, "
consistency": k, "pseudo_FPR_male": fpr, "
female_positive_count": pos_cnt})
566             return pd.DataFrame(rows)
567

```

```

568 # ----- CLI -----
569 def build_argparser():
570     p = argparse.ArgumentParser(description="Q4 Transfer++")
571     p.add_argument("--xlsx", type=str, required=True)
572     p.add_argument("--male-sheet", type=str, default=None)
573     p.add_argument("--female-sheet", type=str, default=None)
574
575     p.add_argument("--col-subject", type=str, default=None)
576     p.add_argument("--col-ga", type=str, default=None)
577     p.add_argument("--col-bmi", type=str, default=None)
578     p.add_argument("--col-z13", type=str, default=None)
579     p.add_argument("--col-z18", type=str, default=None)
580     p.add_argument("--col-z21", type=str, default=None)
581
582     p.add_argument("--qc-features", type=str, default="GA,BMI,
reads,align_rate,dup_rate,unique_reads,gc")
583     p.add_argument("--ga-bins", type=str, default="Early:<=14,
Mid:14-22,Late:>22")
584     p.add_argument("--outdir", type=str, default="
out_q4_transfer_final")
585
586     p.add_argument("--robust-scale", action="store_true")
587     p.add_argument("--standard-scale", dest="robust_scale",
action="store_false")
588     p.set_defaults(robust_scale=True)
589
590     p.add_argument("--student-chrom", type=str, default="21",
help="逗号分隔，需要用 t 分布的染色体（默认 21）")
591     p.add_argument("--winsor", type=float, default=0.01, help=
"对男胎残差的截尾比例（如 0.01 表示 1%）")
592
593     p.add_argument("--batch-col", type=str, default=None, help
="批次列名（检测日期/批号等）；自动数值化或 One-Hot")
594     p.add_argument("--batch-max-levels", type=int, default=20)
595

```

```

596 p.add_argument("--alpha", type=float, default=0.05)
597 p.add_argument("--z-thr", type=float, default=3.0)
598 p.add_argument("--q-thr", type=float, default=0.05)
599 p.add_argument("--suspect-z", type=float, default=2.5)
600 p.add_argument("--suspect-q", type=float, default=0.10)
601 p.add_argument("--min-consistency", type=int, default=2)
602
603 # Per-chromosome overrides
604 p.add_argument("--z-thr-13", type=float, default=None)
605 p.add_argument("--z-thr-18", type=float, default=None)
606 p.add_argument("--z-thr-21", type=float, default=None)
607 p.add_argument("--q-thr-13", type=float, default=None)
608 p.add_argument("--q-thr-18", type=float, default=None)
609 p.add_argument("--q-thr-21", type=float, default=None)
610 p.add_argument("--consist-13", type=int, default=None)
611 p.add_argument("--consist-18", type=int, default=None)
612 p.add_argument("--consist-21", type=int, default=None)
613
614 p.add_argument("--grid-scan", action="store_true", help="
运行阈值网格扫描并输出 FPR 表")
615 p.add_argument("--grid-z", type=str, default="
2.6,2.8,3.0,3.2")
616 p.add_argument("--grid-q", type=str, default="
0.03,0.05,0.08")
617 p.add_argument("--grid-consist", type=str, default="2,3")
618 p.add_argument("--target-fpr", type=float, default=0.01,
help="目标 FPR (用于选择推荐阈值)")
619 return p
620
621 def auto_map_columns(df: pd.DataFrame, overrides: Dict[str,
Optional[str]]) -> Dict[str, Optional[str]]:
622     cand = {
623         "subject_id": ["孕妇代码", "受试者ID", "subject_id", "
ID", "编号"],
624         "GA": ["检测孕周", "检测孕周数", "孕周", "GA", "gw", "

```

```

GA_weeks"],
625     "BMI": ["孕妇BMI", "孕妇BMI指标", "BMI"],
626     "reads": ["原始读段数(估)", "原始读段数", "reads", "
total_reads"],
627     "align_rate": ["在参考基因组上比对的比例", "比对率", "
align_rate"],
628     "dup_rate": ["重复读段的比例", "重复读数比例", "重复率
", "dup_rate"],
629     "unique_reads": ["唯一比对的读段数", "唯一比对读数", "
unique_reads"],
630     "gc": ["GC含量", "GC", "gc"],
631     "Z13": ["13号染色体的Z值", "Z13_prime", "Z13", "Z13'
"],
632     "Z18": ["18号染色体的Z值", "Z18_prime", "Z18", "Z18'
"],
633     "Z21": ["21号染色体的Z值", "Z21_prime", "Z21", "Z21'
"],
634 }
635 out: Dict[str, Optional[str]] = {}
636 for key, cands in cand.items():
637     override = overrides.get(f"col_{key.lower()}")
638     out[key] = override if override is not None else next
((c for c in cands if c in df.columns), None)
639 return out
640
641 def main():
642     ap = build_argparser(); args = ap.parse_args()
643
644     male, female = load_sheets(args.xlsx, args.male_sheet,
args.female_sheet)
645
646     merged_cols = pd.Index(male.columns).union(pd.Index(female
.columns))
647     df_for_map = pd.DataFrame(columns=merged_cols)
648     overrides = {

```

```

649         "col_subject": args.col_subject, "col_ga": args.col_ga
        , "col_bmi": args.col_bmi,
650         "col_z13": args.col_z13, "col_z18": args.col_z18, "
        col_z21": args.col_z21
651     }
652     auto = auto_map_columns(df_for_map, overrides)
653
654     # Rename
655     rename_map_m = {}; rename_map_f = {}
656     for std, real in auto.items():
657         if real is not None:
658             if real in male.columns: rename_map_m[real] = std
659             if real in female.columns: rename_map_f[real] =
std
660     male = male.rename(columns=rename_map_m); female = female.
        rename(columns=rename_map_f)
661
662     # GA to weeks
663     if "GA" in male.columns: male["GA"] = coerce_ga_weeks(male
        ["GA"])
664     if "GA" in female.columns: female["GA"] = coerce_ga_weeks(
        female["GA"])
665
666     # Coerce numerics
667     for name in [auto.get('align_rate'), auto.get('dup_rate'),
        auto.get('gc'),
668                 auto.get('reads'), auto.get('unique_reads'),
        auto.get('BMI')]:
669         for df_ in (male, female):
670             if name and (name in df_.columns):
671                 df_[name] = _coerce_num_series(df_[name])
672
673     # Batch covariate
674     added_batch_feats = []
675     if args.batch_col:

```

```

676         male, feats_m = add_batch_ohe(male, args.batch_col,
max_levels=args.batch_max_levels)
677         female, feats_f = add_batch_ohe(female, args.batch_col
, max_levels=args.batch_max_levels)
678         added_batch_feats = sorted(set(feats_m) | set(feats_f)
)
679
680         # Build QC features
681         qc_feats = [x.strip() for x in (args.qc_features or "").
split(",") if x.strip()]
682         if ("GA" not in qc_feats) and ("GA" in male.columns or "GA
" in female.columns): qc_feats.insert(0,"GA")
683         if ("BMI" not in qc_feats) and ("BMI" in male.columns or "
BMI" in female.columns): qc_feats.insert(1,"BMI")
684         qc_feats = [f for f in qc_feats if f] + added_batch_feats
685         if "intercept" in qc_feats: qc_feats.remove("intercept")
686
687         # Chrom columns
688         chrom_cols = {"13":"Z13","18":"Z18","21":"Z21"}
689         missing_any = [v for v in chrom_cols.values() if (v not in
male.columns or v not in female.columns)]
690         if len(missing_any)>0:
691             warnings.warn(f"[WARN] Z 列缺失: {missing_any}。请用
--col-z13/--col-z18/--col-z21 指定。")
692
693         # Subject id
694         subject_col = "subject_id" if "subject_id" in female.
columns or "subject_id" in male.columns else (auto.get("
subject_id") or "subject_id")
695         if subject_col not in male.columns: male[subject_col] = np
.arange(len(male)).astype(str)
696         if subject_col not in female.columns: female[subject_col]
= np.arange(len(female)).astype(str)
697
698         # Null family per chrom

```

```

699     student_set = set([s.strip() for s in (args.student_chrom
or "").split(",") if s.strip()])
700     null_family_per_chrom = {c: ("student" if c in student_set
else "normal") for c in chrom_cols.keys()}
701     student_df_per_chrom = {c: None for c in chrom_cols.keys()}
702 }
703
704     # Per-chrom thr maps (fallback to global)
705     z_thr_map = {
706         "13": args.z_thr_13 if args.z_thr_13 is not None else
args.z_thr,
707         "18": args.z_thr_18 if args.z_thr_18 is not None else
args.z_thr,
708         "21": args.z_thr_21 if args.z_thr_21 is not None else
args.z_thr,
709     }
710     q_thr_map = {
711         "13": args.q_thr_13 if args.q_thr_13 is not None else
args.q_thr,
712         "18": args.q_thr_18 if args.q_thr_18 is not None else
args.q_thr,
713         "21": args.q_thr_21 if args.q_thr_21 is not None else
args.q_thr,
714     }
715     min_consistency_map = {
716         "13": args.consist_13 if args.consist_13 is not None
else args.min_consistency,
717         "18": args.consist_18 if args.consist_18 is not None
else args.min_consistency,
718         "21": args.consist_21 if args.consist_21 is not None
else args.min_consistency,
719     }
720     fem, subj, cal_df, fpr_val = calibrate_and_apply(
721         male=male, female=female, chrom_cols=chrom_cols,

```

```

722     ga_col="GA", bmi_col=("BMI" if "BMI" in female.columns
723     or "BMI" in male.columns else None),
724     subject_col=subject_col, qc_features=qc_feats,
725     ga_bins_spec=args.ga_bins,
726     outdir=args.outdir, robust_scale=args.robust_scale,
727     min_n_bin=25, winsor_p=args.winsor,
728     null_family_per_chrom=null_family_per_chrom,
729     student_df_per_chrom=student_df_per_chrom,
730     alpha=args.alpha, z_thr=args.z_thr, q_thr=args.q_thr,
731     suspect_z=args.suspect_z, suspect_q=args.suspect_q,
732     min_consistency=args.min_consistency, z_thr_map=
733     z_thr_map, q_thr_map=q_thr_map, min_consistency_map=
734     min_consistency_map,
735     estimate_fpr_with_male=False
736 )
737
738 # Ensure male has GA bins for grid scan
739 if 'ga_bin' not in male.columns and 'GA' in male.columns:
740     male = male.copy()
741     male['ga_bin'] = assign_bins(male['GA'], parse_ga_bins
742     (args.ga_bins))
743
744 # Grid scan if requested
745 if args.grid_scan:
746     z_grid = [float(x) for x in (args.grid_z or "").split(
747     ",") if x.strip()]
748     q_grid = [float(x) for x in (args.grid_q or "").split(
749     ",") if x.strip()]
750     k_grid = [int(x) for x in (args.grid_consist or "").
751     split(",") if x.strip()]
752     grid = grid_scan_thresholds(male, fem, cal_df,
753     chrom_cols, subject_col, "GA", z_grid, q_grid, k_grid,
754     null_family_per_chrom)
755     grid_path = os.path.join(args.outdir, "fpr_grid.csv");
756     grid.to_csv(grid_path, index=False)

```

```

743     gsel = grid[(grid["pseudo_FPR_male"].notna()) & (grid[
"pseudo_FPR_male"] <= args.target_fpr)].copy()
744     if not gsel.empty:
745         best = gsel.sort_values(["female_positive_count",
z_thr", "consistency"], ascending=[False, True, True]).head
(1)
746         best.to_csv(os.path.join(args.outdir, "
fpr_grid_best.csv"), index=False)
747
748     # Print overall with deprecation-safe groupby
749     subj_overall = (
750         subj.groupby("subject_id")["label_subject"]
751         .apply(lambda labs: "阳性" if "阳性" in set(labs)
752             else ("可疑(建议复检)" if any(str(x).
startswith("可疑") for x in labs)
753             else "阴性"))
754         .reset_index(name="overall")
755     )
756     cnts = subj_overall["overall"].value_counts().to_dict()
757     print("[DONE] 个体总体结论: ", cnts)
758     print(f"[INFO] 图表已输出至 {args.outdir}, 并汇总到
figures_report.pdf")
759     if args.grid_scan:
760         print(f"[INFO] 已生成 {args.outdir}/fpr_grid.csv (与
fpr_grid_best.csv, 如可用)")
761
762 if __name__ == "__main__":
763     main()

```